PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

INDIVIDUAL THESIS

Yubitha S
(IT22541284)
Individual component: Dataset Generation
Group ID: R26-CS-010

Report submitted in partial fulfilment of the requirements for the
Bachelor of Science (Honours) Degree in Information Technology
Specialising in Cyber Security

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Yubitha S.
Student ID: IT22541284

Signature: _____ Date: 26/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Anuradha Jayakody		26/10/2026
Co-supervisor: Mr. Dinith Primal		26/10/2026

ABSTRACT

Autonomous vehicle perception models are trained on large sets of labeled road images. Driving simulators such as CARLA avoid recording real people. The generated datasets, however, are usually saved without any protection. Anyone who reaches the storage can change, copy or read them, and teams often share them. Most simulation pipelines focus on realism and give little attention to the security of the data they produce. This thesis presents the Dataset Generation component of a four-part framework for privacy-enhanced synthetic training data. The aim was to build integrity, confidentiality and access control into the generation step and to test them with repeatable checks. The component captured 10,000 CARLA frames in five towns. Each frame has four images (RGB, depth, semantic segmentation and class mask), box labels, and labels for weather, time of day and driving behavior. Every file gets a SHA-256 digest, is checked against a role-based access policy and is encrypted with AES-256. The delivered dataset uses CBC mode, so its integrity rests on the manifests. All 40,000 of its image files decrypted and matched their manifests. A GCM mode was added to detect tampering, and a six-test audit of the GCM engine passed. Decryption restored the data exactly. Thirty encryptions of the same input gave 30 different nonces and ciphertexts. The chi-square statistic of the ciphertext was 272.59, against a 5% critical value of about 293.25. 50.0% of ciphertext bits differed between two encryptions. A modified file was rejected with a MAC check failure. None of 6,093 real encrypted files had a readable header. The analysis of the dataset showed the same weather in all towns and unbalanced driving behaviors. Vehicle_Turning was only 2.0% of the frames. The work shows that a synthetic dataset can be protected when it is created, at a small cost. It also shows that the encryption mode decides how tampering is detected. A secure dataset still needs good scene design to be useful.

Keywords: synthetic data, autonomous vehicles, CARLA, authenticated encryption, role-based access control

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Prof. Anuradha Jayakody, and my co-supervisor, Mr. Dinith Primal, for their guidance and feedback throughout this research. Their advice helped me focus on a security layer that I could build and test in the project time.

I also thank my team members, Priyadarshi S.A.S.D. (IT22309556), Apeksha M.K.S.R (IT22110220) and Wijesinghe R.D.P.T. (IT22066916), for their support and cooperation. The other components decrypted, checked and used my dataset. This helped me improve many design decisions in this thesis. I thank the members of the progress review panels for their questions. These led me to add the security audit and to look at how the plaintext is removed. Finally, I thank the developers of CARLA and PyCryptodome, whose open-source tools made this work possible.

TABLE OF CONTENTS

1 INTRODUCTION **1**
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 2
1.4 Research gap 2
1.5 Aim and objectives 2
1.6 Research questions or hypotheses 3
1.7 Scope limitations and assumptions 3
1.8 Individual contribution 4
1.9 Thesis structure 4
2 LITERATURE REVIEW AND RELATED WORK **5**
2.1 Review method and scope 5
2.2 Domain and theoretical foundations 5
2.3 Related research 6
2.4 Existing systems products and techniques 7
2.5 Gap synthesis and conceptual framework 7
3 RESEARCH METHODOLOGY **10**
3.1 Research design 10
3.2 Data participants or study materials 10
3.3 Experimental or study procedure 12
3.4 Measures and evaluation metrics 13
3.5 Data analysis 13
3.6 Ethics privacy security and responsible AI 14
3.7 Reproducibility and research artefacts 14
4 PRODUCT AND SOLUTION DESIGN **15**
4.1 Requirements 15
4.2 Component architecture within the complete solution 15
4.3 Design alternatives and decisions 16
4.4 Detailed design 17
4.5 Standards and professional practice 20
5 IMPLEMENTATION INTEGRATION AND TESTING **21**
5.1 Development environment and technology stack 21
5.2 Component implementation 21
5.3 Integration with the other components 23
5.4 Verification and software testing 23
5.5 Deployment and operation 24
6 RESULTS AND VALIDATION **26**
6.1 Research results 26
6.2 Product and technical results 29
6.3 User or stakeholder validation 33
6.4 Integration results 33
6.5 Summary of findings 34
7 DISCUSSION **35**
7.1 Interpretation of findings 35
7.2 Comparison with literature baselines and existing solutions 35

7.3 Research contribution 36
7.4 Product contribution 36
7.5 Limitations and threats to validity 36
7.6 Implications 37
8 COMMERCIALISATION AND SUSTAINABILITY 39
8.1 Value proposition and target users 39
8.2 Market and competitive position 39
8.3 Business or sustainability model 39
8.4 Scalability intellectual property and compliance 40
8.5 Contribution to the Sustainable Development Goals 40
9 CONCLUSION AND FUTURE DIRECTIONS 41
9.1 Conclusion 41
9.2 Achievement of objectives 41
9.3 Individual research and product contributions 41
9.4 Future directions 42
REFERENCES 43
GLOSSARY 46
APPENDICES 47
Appendix A Extended results 47
Appendix B Academic integrity and AI use disclosure 49

LIST OF FIGURES

Figure 1.1: Overall system overview of the R26-CS-010 framework 1
Figure 2.1: Conceptual framework of the Dataset Generation component 8
Figure 3.1: Research design and experimental procedure of the component 10
Figure 4.1: Group pipeline architecture and the position of Dataset Generation 16
Figure 4.2: Internal data flow of the Dataset Generation component 17
Figure 4.3: Layouts of the encrypted dataset files in CBC and GCM mode 18
Figure 6.1: Results of audit Tests 2, 3 and 4 and the real folder scan 27
Figure 6.2: Security audit dashboard of the GCM engine (6 of 6 tests passed) 28
Figure 6.3: Weather category and time of day in each town (number of frames) 30
Figure 6.4: Behavior labels over the whole dataset 30
Figure 6.5: Behavior labels by town 31
Figure 6.6: Mean labeled objects per frame and share of frames with no labeled object, by town 32

LIST OF TABLES

Table 2.1: Comparison of AES-256-CBC and AES-256-GCM 6
Table 2.2: Existing systems and techniques compared with the needs of the component 7
Table 2.3: Research gaps addressed by the Dataset Generation component 7
Table 2.4: STRIDE threat classes and their coverage in the component 8
Table 3.1: Composition of the captured dataset by town 11
Table 3.2: Weather and time-of-day settings in every town, compared with the privatized calibration values 11
Table 3.3: Security audit test plan 12
Table 3.4: Measures and evaluation metrics 13
Table 3.5: Reference values for the statistical audit tests 14
Table 4.1: Functional requirements of the Dataset Generation component 15
Table 4.2: Design alternatives and decisions 16
Table 4.3: RBAC role and permission matrix 18
Table 4.4: Threat model and corresponding controls 18
Table 4.5: Security parameters of the implementation 19
Table 4.6: Fields of the town metadata files 19
Table 5.1: Development environment and technology stack 21
Table 5.2: Files exchanged by the Dataset Generation component 23
Table 5.3: Verification of the functional requirements 24
Table 5.4: Test cases of the security audit 24
Table 6.1: Round-trip correctness results 26
Table 6.2: Nonce uniqueness results 26
Table 6.3: Ciphertext byte distribution results 26
Table 6.4: Bit difference results 27
Table 6.5: Tamper detection results 28
Table 6.6: Real capture folder validation results 28
Table 6.7: Summary of audit results for the GCM engine 29
Table 6.8: Integrity evidence for the delivered CBC dataset 29
Table 6.9: Distribution of behavior labels over the dataset 30
Table 6.10: Behavior labels by town (frames and share of the 2,000 frames of each town) 31
Table 6.11: Diversity of behavior labels and empty frames by town 31
Table 6.12: Distribution of labeled objects per frame 32
Table 6.13: Cost of the security layer 32
Table 6.14: Progress review comments on the component, responses and remaining concerns 33
Table 6.15: Integration results reported by the other components 34
Table 7.1: Limitations, their effect and suggested remedies 37
Table 8.1: Main parts of the business model for the component 39
Table 9.1: Achievement of objectives 41
Table A.1: Raw values of the security audit and the manifest check 47
Table A.2: First three records of the Town01 metadata file 47
Table A.3: Frames per weather label and town 47
Table A.4: Number of frames per behavior label and town 48
Table A.5: Expected outcome of each role and permission pair 48
Table A.6: Steps of the security audit 48

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AV	Autonomous Vehicle
CARLA	Car Learning to Act (open-source driving simulator)
CBC	Cipher Block Chaining
CSV	Comma-Separated Values
DP	Differential Privacy
FR	Functional Requirement
GCM	Galois/Counter Mode
GDPR	General Data Protection Regulation
IV	Initialization Vector
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (dataset)
MAC	Message Authentication Code
mAP	Mean Average Precision
NIST	National Institute of Standards and Technology
PNG	Portable Network Graphics
PP	Progress Presentation
RBAC	Role-Based Access Control
RGB	Red, Green, Blue
RQ	Research Question
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
YOLO	You Only Look Once (object detector and label format)

1 INTRODUCTION

1.1 Overall project context

Autonomous vehicles use machine learning to detect objects, follow lanes and decide how to drive. These models need a very large amount of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were made by driving cars with sensors on public roads. These recordings hold personal data. Cameras capture faces and number plates. Repeated routes can show where people live and work. Under the General Data Protection Regulation (GDPR), recordings of identifiable people are personal data. Article 32 requires suitable technical measures to keep such data secure [3].

One common answer is to generate the training data in a driving simulator. CARLA is a popular open-source simulator. It provides towns, traffic, weather and virtual sensors [4]. It was used to build large synthetic datasets such as SHIFT [5]. Synthetic data avoids recording real people. It does not remove the need for security, though. A generated dataset is still a valuable set of files. Anyone who can reach the storage can copy, read, change or delete it. Universities, suppliers and manufacturers also share datasets more and more. Reviews of autonomous driving security list data tampering and unauthorized access as main threats [6], [7].

Project R26-CS-010 deals with these risks. It is a privacy-enhanced synthetic data generation framework with four components [8]. Privacy Integration (IT22309556) applies and tests differential privacy where real information enters the pipeline. Dataset Generation (IT22541284) captures CARLA frames. It protects every file with SHA-256 hashing, AES-256 encryption and role-based access control (RBAC). Utility Evaluation (IT22110220) checks the data for tampering and measures how useful it is for training a detector. Risk and Compliance (IT22066916) checks the evidence with separate code and maps it to GDPR and NIST requirements. Figure 1.1 shows how the four components connect.

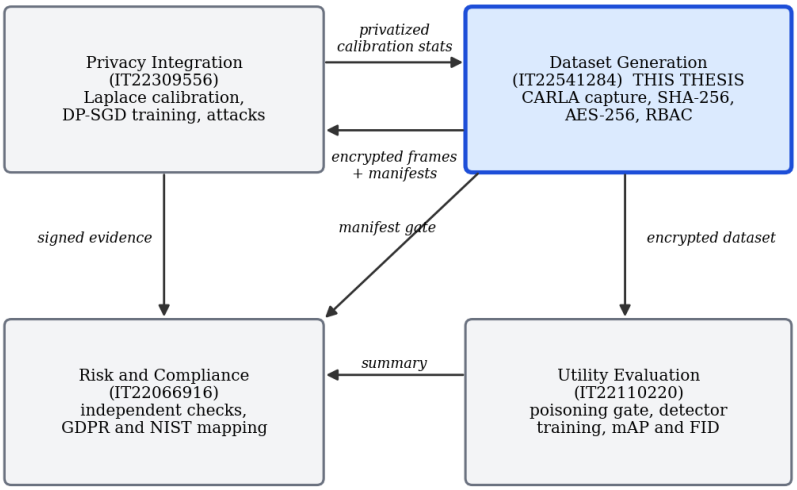


Figure 1.1: Overall system overview of the R26-CS-010 framework

1.2 Individual component context

This thesis covers the Dataset Generation component. This is the part of the framework that produces the data. It runs CARLA in five town maps. For every frame it captures four images: an RGB image, a depth map, a semantic segmentation image and a class mask. Each image is 800 x 600 pixels [8]. It also writes a label file with the boxes of the cars, pedestrians and cyclists in each frame. For every frame it records the weather, the time of day and a driving-behavior label. Every image is protected before it leaves the component. The output is an encrypted dataset of 10,000 frames, which is 40,000 encrypted images. It comes with the label files, a metadata file for each town and a manifest of SHA-256 digests for each town.

The component sits at the start of the data flow. Before capture, it receives privatized calibration statistics from Privacy Integration. These statistics set up the simulator. After capture, it passes the encrypted frames, manifests and metadata to the other components. Privacy Integration uses them for differential-privacy training. Utility Evaluation uses them for decryption, the poisoning gate and detector training. Risk and Compliance uses them for the manifest gate and independent checks [8]. The output of this component is the input of the other three. A weakness here would weaken all of them.

1.3 Problem statement

Most data generation frameworks for simulation focus on realism, sensor quality and variety. Protection of the generated files is often a later step. Where it exists, it is applied after generation [6], [7], [9]. Between capture and protection, raw images sit on disk. They have no integrity check, no encryption and no limit on who can open them. Blurring or anonymizing images protects what the picture shows. It does not stop a file from being changed or read. It also does not control who handles the raw output.

As a result, a team that shares a synthetic dataset cannot prove three things. It cannot prove that the files are unchanged since capture. It cannot prove that they are unreadable without the key. It cannot prove that only authorized roles handled them. Poisoned or damaged training data can also weaken a perception model without anyone noticing [10], [11]. A security layer built into the generation step is therefore needed. It must also be tested in a way that others can repeat.

1.4 Research gap

Earlier work leaves four gaps that matter for this component. First, studies on synthetic driving data focus on realism, variety and domain adaptation. They rarely look at the security of the generation pipeline [5], [12]. Second, security reviews of autonomous driving describe threats and defenses in general. Few of them test a working control inside a data generation pipeline [6], [9]. Third, privacy methods such as differential privacy reduce what the data reveals. They do not protect stored files from tampering or unauthorized reading [13]. Fourth, encryption is often described but rarely tested with measurable checks. A design can use a strong cipher and still fail through nonce reuse or an unauthenticated mode [14], [15]. Section 2.5 explains these gaps in more detail.

1.5 Aim and objectives

The aim of this research is to build a security layer into the synthetic data generation pipeline. It also aims to show, with repeatable tests, that the layer protects the dataset. The group objective for this component is that every generated file is protected [8]. For this thesis, this gives three measurable targets. Every generated file must be hashed, encrypted and covered by an access policy. The security audit tests must pass. No encrypted file may be readable without the key. The specific objectives are:

1. To set up CARLA to capture RGB, depth, semantic segmentation and class-mask images, with box labels, in several town maps.
2. To generate a labeled dataset and describe its coverage of towns, weather, time of day and driving behaviors.
3. To compute a SHA-256 digest for every captured file and keep a manifest for later checks.
4. To encrypt every file with AES-256 before storage or transfer, and to provide an authenticated mode (GCM) that detects modified files.
5. To enforce least-privilege access with an RBAC gateway for three pipeline roles.
6. To test the security layer with an independent audit. The audit covers correctness, nonce uniqueness, ciphertext randomness, ciphertext difference, tamper detection and a scan of real output.
7. To connect the component to the other three components and use the privatized calibration statistics to set up the simulator.

1.6 Research questions or hypotheses

The main research question of this component is:

RQ: Can integrity hashing, authenticated encryption and role-based access control be built into the CARLA data generation pipeline, so that the dataset is protected when it is created? Can tests verify this protection?

The main question was split into three sub-questions:

- **RQ1:** Can every captured file be hashed, encrypted and covered by an access policy at capture time, without changing how CARLA is used?
- **RQ2:** Does the encryption layer pass tests for correctness, semantic security, ciphertext randomness and tamper detection? How is the integrity of the delivered dataset checked?
- **RQ3:** What does the dataset cover across towns, weather, time of day and behaviors? What does the security layer cost in storage and time?

The working hypothesis had four parts. Encrypted files would decrypt exactly. Identical inputs would never give identical ciphertexts. The ciphertext would look random in statistical tests. Any change to a protected file would be detected.

1.7 Scope limitations and assumptions

The component protects dataset files when they are generated and while they are stored. It does not cover differential privacy, model training or attack testing. These belong to the other components. It does not protect the simulation computer, data in memory, or the encryption key itself beyond the limits in Section 7.5. Network security and user authentication are also outside the scope.

The work makes four assumptions. The simulation computer and the pipeline code are trusted. The storage that holds the dataset is not fully trusted, and neither are the people who can reach it. All frames come from CARLA, so no real person, vehicle or place is recorded. Privacy Integration computed the calibration statistics from the Waymo Open Dataset. It privatized them with the Laplace mechanism before this component used them [8].

1.8 Individual contribution

The author designed and built the whole Dataset Generation component. The main contributions are:

- A multi-town CARLA capture pipeline. One script and a table of town profiles drive it. It produced 10,000 frames in five towns. Each frame has four images, a box label file and labels for behavior, weather and time of day.
- A `SecurityEngine` class that computes SHA-256 digests in blocks and encrypts every file with AES-256. The delivered dataset uses CBC mode. A GCM mode was added to detect tampering.
- An `RBACGateway` class with deny-by-default permissions for the Simulation Engineer, Privacy Engineer and ML Researcher roles.
- A six-test security audit and 26 unit tests that can be repeated without the simulator. A manifest that the other components use to check the data.
- A dataset analysis of coverage by town, weather, time of day, behavior and objects per frame.
- The files, metadata and interfaces that connect the component to the other three components.

1.9 Thesis structure

Chapter 2 reviews related work and builds the conceptual framework. Chapter 3 explains the research design, data, procedure and metrics. Chapter 4 presents the requirements and the design of the component. Chapter 5 covers its implementation, integration and testing. Chapter 6 reports the results. Chapter 7 discusses them. Chapter 8 looks at commercialization and sustainability. Chapter 9 concludes the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covered four topics that the component depends on. These were synthetic data for autonomous driving, security threats to autonomous driving data, the cryptography and access control used to protect stored data, and the link between security and privacy for synthetic data. The sources are mainly peer-reviewed papers and standards. They include CVPR, USENIX, EUROCRYPT and IEEE papers and NIST publications. Security surveys were checked against their published records. A work was included if it described a method used in this component, tested synthetic driving data, or studied threats to autonomous driving data.

2.2 Domain and theoretical foundations

Synthetic data for autonomous driving

Real benchmark datasets such as KITTI [1] and Cityscapes [16] gave the field labeled urban scenes. Real data collection is costly and raises privacy concerns. Researchers therefore used virtual worlds to make labeled data cheaply. Examples are SYNTHIA [17] and the CARLA simulator [4]. AirSim [18] is a similar platform. CARLA provides several town maps, traffic, pedestrians, weather presets and sensors such as RGB and depth cameras. It also gives automatic labels. Synthetic data can also show rare situations that are hard or unsafe to record on real roads. Domain randomization is one way to reduce the gap between simulated and real images [19]. A survey of synthetic data for deep learning describes the benefits and the open problems [12]. The SHIFT dataset was built entirely in CARLA. It shows that simulation-based generation can reach a large scale [5].

Threats to autonomous driving data

Gao et al. surveyed the security of autonomous driving. They described attack surfaces in sensors, operating systems, control systems and vehicle communication. Data integrity and confidentiality were among the security properties they examined [6]. Liao et al. studied data security from technical, legal and ethical sides. They pointed out that law lags behind technology and that breaches by insiders are a real risk [7]. Lippi et al. reviewed attack types and defenses, including privacy-preserving designs [9]. Xie et al. surveyed the privacy of autonomous vehicles and the protection methods available [13]. These works agree that stored AV data needs integrity, confidentiality and access control. A synthetic data pipeline is no different.

Training data can also be attacked directly. Poisoning attacks change training examples so that a model learns the wrong behavior [10]. Backdoor attacks hide a trigger in the data supply chain [11]. These attacks do not need access to the model. They only need the ability to change files. This is why detecting file changes matters for a dataset that is stored and shared.

Hashing, encryption and authenticated encryption

SHA-256 is a hash function from the SHA-2 family. It turns any input into a 256-bit digest [20]. A change of one bit in the input gives a completely different digest. Finding two inputs

with the same digest is not feasible. If the digest of each file is stored at capture time, it works as a fingerprint. It can be recomputed and compared later. A hash shows whether a file has changed. It does not hide the content, and it does not show who made the file.

The Advanced Encryption Standard (AES) is the standard block cipher. It works on 128-bit blocks with keys of 128, 192 or 256 bits [21]. A mode of operation is needed to encrypt files longer than one block. The choice of mode matters. Cipher Block Chaining (CBC) [22] gives confidentiality only. A modified ciphertext can decrypt to damaged data without any error. CBC implementations that reveal padding errors can also be attacked [23]. Detecting changes therefore needs another mechanism, such as a hash manifest. Combining encryption and authentication correctly is a known source of mistakes [24].

Galois/Counter Mode (GCM) is an authenticated-encryption mode. It encrypts with counter mode. It also computes an authentication tag over the ciphertext with a Galois-field function [25], [26]. During decryption, the tag is checked first. Any change to the ciphertext, nonce or tag makes decryption fail. GCM has one strict rule. A nonce must never be reused under the same key. Reuse breaks both confidentiality and authenticity. Practical forgery attacks on GCM through nonce reuse have been shown in real systems [14], [15]. For this reason, nonce uniqueness must be tested.

Table 2.1 compares the two modes. It explains why this component supports both. The project proposal specified CBC, and it is simple. GCM detects tampering by itself.

Table 2.1: Comparison of AES-256-CBC and AES-256-GCM

Property	AES-256-CBC	AES-256-GCM
Confidentiality	Yes	Yes
Built-in tamper detection	No (needs a separate hash or MAC)	Yes (128-bit authentication tag)
Needs padding	Yes (PKCS7)	No (counter-mode stream)
Random IV or nonce	Yes, 16-byte IV, unpredictable	Yes, must never repeat for a key
Storage overhead per file	16-byte IV plus 1 to 16 bytes of padding	16-byte nonce plus 16-byte tag (32 bytes)
Result of a modified file	May decrypt to damaged data without an error	Decryption fails with a MAC error

Access control

Role-based access control (RBAC) gives permissions to roles instead of to single users. Users get permissions through the roles they hold [27]. It supports two classic design rules. Least privilege gives each role only the permissions it needs for its duties. Deny by default refuses anything that is not clearly allowed [28]. In a data pipeline, RBAC separates the people and programs that create data, those that process it and those that only use the result.

2.3 Related research

Synthetic data generation. Most work on synthetic driving data aims to make the data realistic or to close the gap to real images. Studies such as SHIFT [5] and SYNTHIA [17] describe the simulator setup and the labels. They do not describe how the generated files are protected. The

CARLA paper [4] does not discuss integrity checks, encryption or access control for the data that the simulator produces.

Privacy of synthetic data. Stadler et al. showed that synthetic data does not protect the records it came from, unless a formal privacy method is used [29]. Differential privacy is such a method [30], [31]. It protects the information in the data. This is a different control from the one studied here. A privacy-protected dataset can still be changed or copied if its files are not secured. The two controls work together.

Security of autonomous driving data. The security surveys [6], [7], [9] describe the threats in broad terms. They recommend encryption, integrity protection and access control. They do not test these controls in a synthetic data pipeline. Liao et al. also stress that legal and ethical duties around data security are growing [7]. This supports building protection into the pipeline from the start.

Testing encryption in practice. Cryptographic standards define what a correct implementation must do [21], [26]. Statistical test suites exist to check randomness [32]. Studies of real failures show that the mode and the nonce handling cause most problems, not the cipher itself [15], [23]. A practical test of a data pipeline should therefore check correctness, nonce uniqueness, randomness and tamper detection. This thesis does this.

2.4 Existing systems products and techniques

Several tools and practices cover parts of the problem. Table 2.2 compares them with the needs of this component.

Table 2.2: Existing systems and techniques compared with the needs of the component

System or technique	What it provides	Limitation for this work
CARLA, AirSim and similar simulators [4], [18]	Labeled synthetic sensor data	No protection of generated files; no integrity record or access policy
Public datasets such as KITTI, Waymo and SHIFT [1], [2], [5]	Large benchmark data and labels	Distributed as plain files; protection is left to the user
Image blurring and anonymization	Removes visible identifiers from images	Does not stop tampering or unauthorized reading of files
Differential privacy methods [30]	Formal limit on what a model reveals	Protects information content, not file integrity or access
Disk or cloud storage encryption	Encrypts data at rest	Applied after generation; no per-file integrity manifest or role policy for the pipeline

Disk and cloud encryption are the closest practical alternative. They protect stored data, but they work on the storage and not on the generation step. They do not record a digest for each captured frame. They do not apply a role policy to pipeline operations. They give no per-file evidence that can be shared with another team. The component in this thesis applies the controls to each file at the moment of capture.

2.5 Gap synthesis and conceptual framework

Table 2.3 summarizes the gaps that this component addresses and the approach for each.

Table 2.3: Research gaps addressed by the Dataset Generation component

Gap	Limitation of existing work	Approach in this component
Pipeline security	Synthetic data work focuses on realism; security is left to later steps [5], [6]	Controls built into capture: hash, check role, encrypt
Tamper detection	Unauthenticated modes such as CBC cannot detect changes alone [23]	SHA-256 manifest for every file, plus a GCM mode with a per-file tag
Measured evidence	Encryption is described but rarely tested [15]	Six-test audit of the engine and a scan of real output
Least privilege	Access control is recommended but not applied to pipeline roles [7]	RBAC gateway with deny-by-default permissions for three roles

These gaps lead to the conceptual framework in Figure 2.1. Each threat to the dataset has a matching control. Each control produces evidence, and a separate check tests that evidence. A SHA-256 manifest handles tampering, and in GCM mode the authentication tag helps too. AES-256 encryption handles disclosure. The RBAC gateway handles privilege misuse. Protection is only claimed when the evidence from the audit and from the other components agrees.

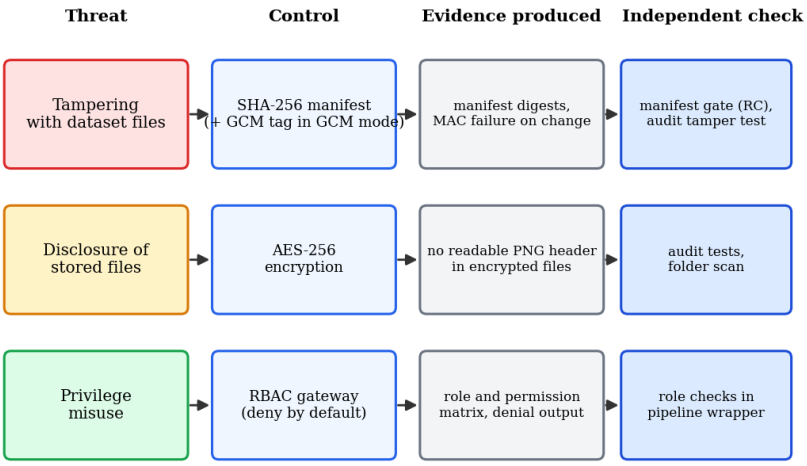


Figure 2.1: Conceptual framework of the Dataset Generation component

The conceptual framework can be checked against a standard threat list. STRIDE groups threats into spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege [33]. It helps a designer see which threats a design covers and which it leaves open. Table 2.4 applies it to the dataset pipeline. The component covers tampering, information disclosure and elevation of privilege well. It covers spoofing only in part. It leaves repudiation and denial of service open. These open items appear in the limitations in Section 7.5 and the future work in Section 9.4.

Table 2.4: STRIDE threat classes and their coverage in the component

STRIDE class	Example in a dataset pipeline	Coverage in this component
Spoofing	A user claims a role they do not hold	Partly: roles are checked, but users are not authenticated
Tampering	Images or labels are changed in storage	Covered: SHA-256 manifest and manifest gate; GCM tag in GCM mode
Repudiation	Nobody can show who read or changed a file	Not covered: no persistent access log
Information disclosure	Dataset files are copied from storage	Covered: AES-256 encryption; plaintext deleted after encryption
Denial of service	Files are deleted or the storage is filled	Not covered: the manifest shows missing files but does not prevent loss
Elevation of privilege	An ML user reads raw simulation output	Covered: deny-by-default RBAC policy

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a design science approach with quantitative evaluation. A security artefact was built first and then tested in controlled experiments. The design had four parts:

- **Build and generate.** The capture pipeline and the security layer were built. The dataset was generated in five CARLA towns.
- **Isolated testing.** The security classes contain no simulator code, so the audit runs them on their own. Each test checks one property and changes only one factor.
- **Real-output check.** After the controlled tests, the audit scanned a real encrypted capture folder. This confirms that real output is protected.
- **Integration check.** The other three components decrypted, verified and used the dataset. This would show problems at the interfaces [8].

Figure 3.1 shows the research process from the problem to the conclusions. The audit results fed back into the design. The proposal specified CBC, and the dataset was captured with CBC. The audit then showed that CBC cannot detect a modified file on its own. A GCM mode was therefore added to the engine and tested. The manifests became the integrity check for the CBC dataset [8].

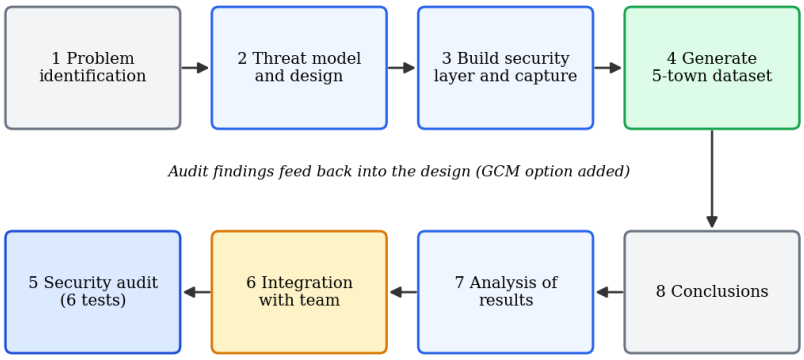


Figure 3.1: Research design and experimental procedure of the component

3.2 Data participants or study materials

No human participants took part in this study. The study material was synthetic data generated in CARLA.

Towns and frames. The dataset was captured in five CARLA town maps, Town01 to Town05. Each town has 2,000 frames, so the dataset has 10,000 frames. Every frame has four images (RGB, depth, semantic segmentation and class mask) of 800 x 600 pixels. The dataset therefore holds 40,000 encrypted images [8]. In each town, consecutive frame numbers in the metadata differ by exactly 16 simulation frames. Frames were therefore taken at a regular interval. Table 3.1 describes each town.

Table 3.1: Composition of the captured dataset by town

Town	Frames	Encrypted images	Labeled objects	Mean objects per frame	Frames with no object
Town01	2,000	8,000	1,481	0.74	970 (48.5%)
Town02	2,000	8,000	1,926	0.96	861 (43.1%)
Town03	2,000	8,000	3,066	1.53	436 (21.8%)
Town04	2,000	8,000	4,093	2.05	668 (33.4%)
Town05	2,000	8,000	4,518	2.26	329 (16.5%)
Total	10,000	40,000	15,084	1.51	3,264 (32.6%)

Town maps. The five towns have different layouts. According to the CARLA documentation [34], Town01 is a small town with a river and several bridges. Town02 is a small residential town. Town03 is a larger urban map with a roundabout and roads at different heights. Town04 is a small town in the mountains with a long highway. Town05 is an urban map with a square grid of cross junctions and multilane roads. These differences give the dataset variety in road structure.

Labels. Each frame has a label file in YOLO format. Each line holds a class and the normalized center, width and height of one box. There are three classes: car, pedestrian and cyclist. The metadata records the number of labeled objects per frame. The dataset has 15,084 labeled objects in total. 3,264 frames (32.6%) have none. Each frame also has a driving-behavior label for the ego vehicle. The labels are Standard_Flow, Heavy_Braking, Intersection_Nav and Vehicle_Turning.

Weather and time of day. Each frame has a weather category (clear, cloudy or rain) and a time of day (morning, afternoon, evening or night). Table 3.2 shows that the counts are the same in all five towns. The simulator was set up from the same privatized calibration statistics for every town. This fixed the share of each weather and time-of-day value [8]. No frame is labeled fog, because the privatized value for fog was zero.

Table 3.2: Weather and time-of-day settings in every town, compared with the privatized calibration values

Setting	Frames per town	Share of each town	Value in the privatized calibration
Weather: clear	1,248	62.4%	0.624
Weather: cloudy	270	13.5%	0.135
Weather: rain	482	24.1%	0.241
Weather: fog	0	0.0%	0.000
Time: morning	288	14.4%	0.144
Time: afternoon	230	11.5%	0.115
Time: evening	1,149	57.5%	0.575
Time: night	333	16.7%	0.167

[CONFIRM before submission: in Table 4.3 of the group report these exact values appear in the earlier calibration release (total epsilon 12.6), while the corrected release (total epsilon 1.8)

gives clear 0.650, cloudy 0.156, rain 0.195 and evening 0.837. Check with Privacy Integration which release configured the delivered capture, and correct this thesis or the group report so that they agree.]

Calibration statistics. The simulator is set up from seven aggregate statistics. These are vehicle and pedestrian density per kilometer, average vehicle and pedestrian speed, intersections per kilometer, and the weather and time-of-day distributions. Privacy Integration computed them from the Waymo Open Dataset. It added Laplace noise within a total budget of epsilon 1.8. It then passed them on in `calibration_stats.json` [8]. Only the privatized values were used to set up the simulator.

Security test material. The first five audit tests use files of pseudo-random bytes between 4 KB and 200 KB. They use a random key that exists only in memory. The sixth test scans a real capture folder with the project key and writes no plaintext [8]. The real folder was a separate Town01 capture of 6,093 encrypted images made with the GCM engine.

3.3 Experimental or study procedure

The procedure had six steps.

1. **Configuration.** CARLA was started in the chosen town with the settings from the town profile. The profile holds the privatized calibration values.
2. **Capture.** The ego vehicle drove through the town. The four images and the box labels were recorded for every sampled frame.
3. **Protection.** For every captured file, the pipeline computed a SHA-256 digest and checked the role against the RBAC policy. It then encrypted the file with AES-256, wrote the `.enc` file and removed the plaintext.
4. **Metadata.** One row was written to the town metadata file for each frame. It holds the file names, object count, weather, time of day, behavior label, town and security field.
5. **Audit.** The audit ran five tests on the engine with synthetic bytes. It then scanned a real encrypted folder.
6. **Analysis and hand-over.** The metadata of all five towns was analyzed for coverage. The encrypted dataset, manifests and metadata were passed to the other components.

The six audit tests are listed in Table 3.3. Each test was designed to fail if a specific weakness exists. The audit runs on the GCM mode of the engine, because only GCM has a tag that can be tested for tampering. For CBC, the audit reports the tamper test as not applicable [8].

Table 3.3: Security audit test plan

No.	Test	What it checks	Pass criterion
1	Round-trip correctness	Decryption restores the original data	Recovered bytes identical to the original
2	Nonce uniqueness	Same input never gives the same output	30 encryptions give 30 distinct nonces and ciphertexts
3	Byte distribution	Ciphertext looks random	Chi-square statistic below the 5% critical value (df = 255)

No.	Test	What it checks	Pass criterion
4	Bit difference	Near-identical inputs give unrelated ciphertexts	About 50% of ciphertext bits differ
5	Tamper detection	A changed file is rejected	MAC check fails and no output file is written
6	Real folder scan	Real output is not readable	No encrypted file starts with the PNG signature

For Test 1, a 4 KB file was encrypted and decrypted. For Test 2, one plaintext was encrypted 30 times with the same key. For Test 3, a 200,000-byte file was encrypted. The number of times each of the 256 byte values appears in the ciphertext was then counted. For Test 4, two 4 KB inputs that differ in exactly one bit were encrypted. Their ciphertexts were compared bit by bit. For Test 5, one byte of an encrypted file was changed with an XOR of 0xFF, and decryption was attempted. For Test 6, the first eight bytes of every encrypted file in the real folder were compared with the PNG signature.

3.4 Measures and evaluation metrics

Table 3.4 lists the measures used in the evaluation.

Table 3.4: Measures and evaluation metrics

Measure	Definition	Purpose
Round-trip match	Recovered bytes equal original bytes	Correctness of encryption
Unique nonces and ciphertexts	Count of distinct values in 30 runs	Semantic security
Chi-square statistic	Sum over 256 byte values of (observed - expected)^2 / expected	Randomness of ciphertext
Bit difference	Share of ciphertext bits that differ between two encryptions	Unrelatedness of ciphertexts
Tamper result	Whether a modified file is accepted or rejected	Authenticity (GCM tag)
Readable-header count	Encrypted files starting with the PNG signature	Protection of real output
Manifest match	Decrypted files whose SHA-256 equals the manifest entry	Integrity of the delivered dataset
Encryption and decryption time	Milliseconds for a 4 KB file, including file input and output	Cost of the security layer
Storage overhead	Extra bytes per encrypted file	Cost of the security layer
Coverage counts	Frames per town, weather, time of day and behavior; objects per frame	What the dataset contains

3.5 Data analysis

The security results are reported as direct measurements. The chi-square statistic for Test 3 is calculated as follows. Let N be the number of ciphertext bytes. Let O(i) be the number of times byte value i appears, for i from 0 to 255. If the bytes are uniform, each value is expected E = N / 256 times. The statistic is the sum over all 256 values of (O(i) - E)^2 / E. For N = 200,000, the expected count is E = 781.25. When the bytes are random, the statistic follows a chi-square

distribution with 255 degrees of freedom. Its mean is 255, and its standard deviation is about 22.6. A single byte count has a standard deviation of about 27.9. Table 3.5 lists the reference values used for comparison. Tests 1, 2, 5 and 6 are pass or fail tests. Test 4 is judged against the 50% value that unrelated random data would give.

Table 3.5: Reference values for the statistical audit tests

Quantity	Reference value for random data
Expected count per byte value (N = 200,000)	781.25
Standard deviation of one count	about 27.9
Mean of the chi-square statistic (df = 255)	255
Standard deviation of the statistic	about 22.6
5% critical value (df = 255)	about 293.25
Expected share of differing bits between two unrelated ciphertexts	50% (16,384 of 32,768 bits)
Standard deviation of the differing-bit count	about 90.5 bits

The dataset was analyzed with pandas and matplotlib. The metadata files of the five towns were loaded into one table of 10,000 rows. Counts were then calculated for each town, weather category, time of day and behavior label. The number of labeled objects per frame was also counted. A normalized Shannon entropy of the behavior labels was calculated for each town to compare diversity. These analyses show what the dataset covers and where it is unbalanced.

3.6 Ethics privacy security and responsible AI

All images in this work were generated inside the CARLA simulator. No real person, vehicle or place was recorded, and no personal data was collected. The component still treats the dataset as sensitive. The aim of the project is to build a pipeline that would be safe to use with real data.

The component does not claim legal compliance. It produces technical evidence. This is a hash manifest, encrypted files and test results. A data controller can use this evidence to decide whether a dataset meets its duties under GDPR Article 32 [3]. Known weaknesses, such as the limits of key handling, are stated in this thesis. Before the full experiment, a group code review found the AES key and the HMAC key written into source files. Both keys were replaced. They were moved to local environment files that are kept out of version control. A regression test now fails if an old key appears in the source again [8].

Generative AI was used to help prepare this thesis. Appendix B describes the use. The author remains responsible for the design, the experiments, the numbers and the conclusions.

3.7 Reproducibility and research artefacts

The audit runs the real `SecurityEngine` class. It creates its own test files from fixed random seeds and writes the numeric results to a file. Another person can therefore repeat the first five tests without CARLA or the dataset. The sixth test needs a real encrypted folder and the project key. The dataset analysis reads the five town metadata files and recreates the coverage figures

and tables. The main research artefacts are listed below. They are the capture and security code with its 26 unit tests, the encrypted dataset with its manifests and metadata files, the audit and its results, and the analysis scripts.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from the project proposal, the needs of the other three components and the stakeholders named in the proposal. The stakeholders were autonomous vehicle researchers and AI developers, who need training data they can trust. They also included simulation engineers who run the generator, cybersecurity engineers who audit the controls, and companies that want to share synthetic data safely. Table 4.1 lists the functional requirements of this component.

Table 4.1: Functional requirements of the Dataset Generation component

ID	Requirement	Source
FR-01	Generate driving scenes with CARLA in several towns	Proposal FR1, FR2
FR-02	Capture RGB, depth, semantic and class-mask images with box labels and behavior labels	Proposal FR3; needs of Utility Evaluation
FR-03	Compute a SHA-256 digest of every generated file and keep a manifest	Proposal FR4
FR-04	Encrypt every file with AES-256 before storage or transfer	Proposal FR5
FR-05	Enforce role-based access on pipeline operations	Proposal FR6
FR-06	Store the dataset in a structured format with metadata	Proposal FR8
FR-07	Use the privatized calibration statistics to configure the simulator	Needs of Privacy Integration
FR-08	Detect modified files before they are used	Threat model (tampering)

The non-functional requirements were:

- **Security.** Encrypted files must be unreadable without the key. Changes must be detected.
- **Performance.** Hashing and encryption must not delay capture in a noticeable way.
- **Testability.** The security code must be testable without the simulator.
- **Interoperability.** Other components must be able to decrypt the files and read the metadata, labels and manifests.
- **Scalability.** The design must support more towns, weather settings and sensors without changes to the security layer.

4.2 Component architecture within the complete solution

Figure 4.1 shows where the component sits in the group pipeline. It works at the start of the data flow. It receives `calibration_stats.json` from Privacy Integration. It runs the capture and protection pipeline in the five towns. It then gives the encrypted frames, manifests and metadata files to the other three components. Every file is encrypted and listed in a manifest. Each receiving component can therefore check the data before it uses it.

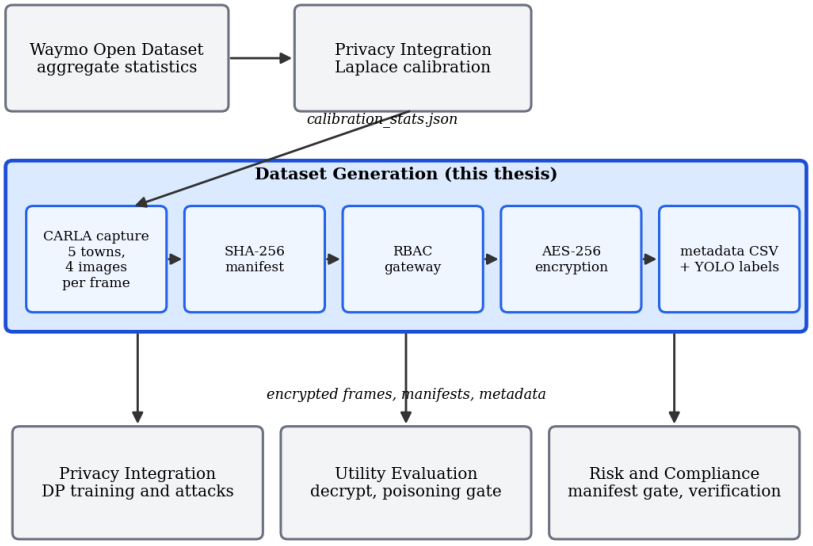


Figure 4.1: Group pipeline architecture and the position of Dataset Generation

This position gives the component a clear boundary. The encrypted dataset and its manifests are the only path from the simulator to the rest of the project. All access to raw output passes through the RBAC check. The boundary can therefore be audited in one place.

4.3 Design alternatives and decisions

Table 4.2 lists the main design choices, the alternatives that were considered and the reason for each choice.

Table 4.2: Design alternatives and decisions

Decision	Alternative	Reason for the choice
Deliver the dataset in AES-256-CBC, with a GCM mode added to the engine	GCM only; CBC only	CBC was specified in the proposal and was used for the capture. The audit then showed that CBC cannot detect changes [22], [23], so GCM [26] was added and tested. The delivered dataset relies on the manifests
Random 16-byte IV or nonce for every file	Counter or fixed value	A random value for every file makes equal images give different ciphertexts and avoids reuse without shared state
Per-file encryption	One archive per batch	Each file can be verified, decrypted and replaced on its own, and one damaged file does not block the batch
SHA-256 manifest of the plaintext, one per town	No manifest (rely on the GCM tag only)	The manifest identifies the original image, works for CBC files and lets another component confirm that a decrypted file is the one that was captured
Write the encrypted copy first, then delete the plaintext	Keep both copies	A readable copy would defeat the encryption, and writing first avoids losing data if encryption fails
Deny-by-default RBAC	Allow-by-default with blocked actions	Unknown roles and unknown actions are refused, which follows least privilege [28]
Security classes independent of CARLA	Security code inside the capture script	Allows the same code to be tested in the audit and in 26 unit tests
Streaming hash in 4,096-byte blocks	Read the whole file into memory	Constant memory use for any file size

Decision	Alternative	Reason for the choice
Key from a local environment file, error if missing	Random key if none is given	A random key would make the dataset unrecoverable, so the final engine refuses to run without a key

The first decision shaped the rest of the design and how the results must be read. The delivered dataset is encrypted with CBC. Its integrity therefore depends on the SHA-256 manifests and on the manifest gate that checks them [8]. The audit tested the GCM mode, and it was used for a separate Town01 capture. Moving the delivered dataset to GCM is the first file-protection item of future work in Section 9.4.

4.4 Detailed design

The component is written in Python. Figure 4.2 shows how a captured frame moves through it.

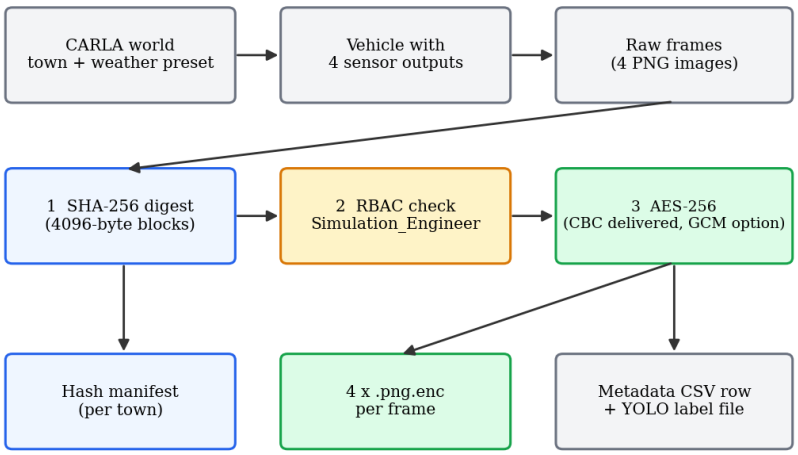


Figure 4.2: Internal data flow of the Dataset Generation component

Capture module. One capture script serves all five towns. A table of profiles keeps the differences between towns, including the privatized calibration values. CARLA is started in the chosen town. A vehicle with the sensors drives through the town, and frames are saved at a fixed interval. The module writes the four images and the box label file of each frame. It assigns the behavior, weather and time-of-day labels and writes one metadata row per frame.

Integrity module. The method `generate_sha256` reads each file in 4,096-byte blocks and returns the SHA-256 digest as a hexadecimal string. The digests of the original plaintext files form the manifest of the town. A mismatch found later marks the file as possibly changed.

Encryption module. The class `SecurityEngine` holds a 32-byte key and has two modes. CBC mode protects the delivered dataset. It creates a random 16-byte IV for each file and encrypts the padded image. The IV is written first, followed by the ciphertext. GCM mode creates a fresh 16-byte nonce for each file. It writes the nonce, a 16-byte tag and the ciphertext.

In both modes, the encrypted copy is saved with the extension `.enc`. The plaintext file is deleted only afterward. Figure 4.3 shows the two file layouts.

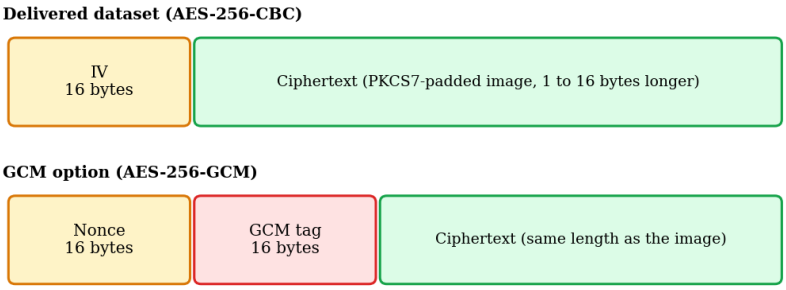


Figure 4.3: Layouts of the encrypted dataset files in CBC and GCM mode

RBAC module. The class `RBACGateway` holds a dictionary that maps each role to its allowed operations. A request is granted only if the role exists and the operation is in its list. Table 4.3 shows the policy.

Table 4.3: RBAC role and permission matrix

Role	Permitted operations	Purpose
Simulation_Engineer	create, verify, encrypt	Produces and protects raw simulation output
Privacy_Engineer	verify	Checks integrity before privacy processing
ML_Researcher	read_processed	Uses processed datasets only

Threat model. The design assumes that the simulation computer is trusted and the storage is not. An attacker can read files from storage, change files in storage, or try to use a role without the needed permission. Table 4.4 maps each threat to its control and to the evidence in Chapter 6.

Table 4.4: Threat model and corresponding controls

Threat	Example	Control	Evidence
Tampering	Images in a batch are replaced	SHA-256 manifest per town; GCM tag in GCM mode	Manifest gate; Test 5
Disclosure	Dataset files copied from storage	AES-256 encryption; plaintext deleted	Tests 1 to 4; folder scan
Privilege misuse	An ML user reads raw output	RBAC deny by default	Role and permission matrix
Output linkage	Equal images show equal ciphertext	Fresh random IV or nonce per file	Test 2
Silent corruption	Damaged file used for training	Manifest check; decrypt_and_verify in GCM mode	Manifest gate; Test 5

Security parameters

Table 4.5 lists the parameters of the security layer as implemented.

Table 4.5: Security parameters of the implementation

Parameter	Value
Hash function	SHA-256, 256-bit digest (64 hexadecimal characters), file read in 4,096-byte blocks
Cipher and key	AES with a 256-bit (32-byte) key, read from a local environment file
Delivered dataset (CBC)	Random 16-byte IV, then ciphertext with PKCS7 padding; no authentication tag
GCM mode	Random 16-byte nonce, then 16-byte tag, then ciphertext; tag checked before any output
Overhead per file	CBC: 16 bytes of IV plus 1 to 16 bytes of padding. GCM: 32 bytes
Plaintext handling	The encrypted copy is written first, then the plaintext file is deleted
Key rule	A missing or wrong-length key raises an error; no random key is created

Operation sequence for one file

Every captured file goes through the same short sequence of operations. This makes the pipeline easy to audit.

1. CARLA writes the raw PNG image to disk.
2. The pipeline asks the RBAC gateway whether the active role may create and encrypt data.
3. The SHA-256 digest of the file is computed and added to the manifest.
4. The file is encrypted with a fresh random IV or nonce.
5. The `.enc` file is written.
6. The plaintext file is deleted.
7. A record is added to the town metadata file when all four images of the frame are done.

Metadata schema

Each town has a metadata file in CSV format, with one row for each frame. Table 4.6 lists the fields. Frame numbers repeat across towns. A town prefix is therefore added to every file name when the towns are merged [8].

Table 4.6: Fields of the town metadata files

Field	Type	Meaning and example
RGB_File	Text	Encrypted RGB image, for example rgb_f000224.png.enc
Depth_File	Text	Encrypted depth map
Semantic_File	Text	Encrypted semantic segmentation image
ClassMask_File	Text	Encrypted class mask
BBox_File	Text	Box label file in YOLO format, for example bbox_f000224.txt
Num_Objects	Integer	Number of labeled cars, pedestrians and cyclists in the frame
Weather_Category	Label	clear, cloudy or rain
Time_Of_Day	Label	morning, afternoon, evening or night
Weather_Label	Label	Combination of the two, for example rain_evening

Field	Type	Meaning and example
Event_Type	Label	Driving behavior of the ego vehicle, for example Standard_Flow
Town	Label	CARLA map, Town01 to Town05
Security	Label	SHA256+AES256 for every record

The metadata files contain only file names, counts and labels. They do not contain image content. They can therefore be shared with the roles that need to select data, without showing the images. The label files are plain text, so the other components can read them directly.

4.5 Standards and professional practice

The design follows the standards and practices below.

- **Data protection by design.** Protection is applied when the data is created, not afterward. GDPR Article 32 asks for suitable technical measures such as encryption. It also asks for the ability to keep data integrity and confidentiality [3].
- **Cryptographic standards.** Hashing uses SHA-256 as defined in FIPS 180-4 [20]. Encryption uses AES as defined in FIPS 197 [21]. CBC mode follows NIST SP 800-38A [22], and GCM mode follows NIST SP 800-38D [26].
- **Least privilege.** Role permissions follow the rule that each role gets only what its duties need [28], [27].
- **Key management guidance.** NIST SP 800-57 states that keys should be protected separately from the data they protect [35]. Section 7.5 discusses how far the current implementation meets this.
- **Engineering practice.** The security code is separate from the simulator code and covered by unit tests. It has no CARLA dependencies. A continuous integration job in the shared repository blocks keys and datasets from being committed [8].

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

All components of the project were written in Python and kept in one shared repository [8]. Table 5.1 lists the tools that this component used and their purpose. All of them are free and open-source.

Table 5.1: Development environment and technology stack

Tool	Purpose
CARLA Simulator [CARLA version], Python API	Towns, weather, vehicle and the four image sensors
Python	Capture script, security classes, audit and analysis
PyCryptodome [36]	AES encryption and decryption in CBC and GCM mode
Python hashlib	SHA-256 digests
NumPy	Byte-frequency counts for the chi-square test
pandas and matplotlib	Loading the town metadata files, coverage tables and charts
GitHub Actions	Commit checks and tests on every push to the shared repository [8]
Hardware	CARLA capture machine [GPU model, CPU, RAM and storage]

5.2 Component implementation

The security layer is made of two Python classes. They do not depend on CARLA, so the same code runs in the capture script and in the audit. One capture script serves all five towns. A table of profiles keeps the differences between towns [8].

Hashing. The method `generate_sha256` reads a file in 4,096-byte blocks and updates a SHA-256 object. The final digest is returned as a hexadecimal string. It is stored in the manifest of the town.

```
def generate_sha256(self, file_path):
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b""):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

Encryption of the delivered dataset. The first version of the engine used AES-256 in CBC mode. The method `encrypt_data` creates a CBC cipher with a random IV and encrypts the padded file contents. It writes the IV followed by the ciphertext to a `.enc` file and then deletes the plaintext. This version produced the delivered dataset.

```
def encrypt_data(self, file_path):
    cipher = AES.new(self.key, AES.MODE_CBC)
    with open(file_path, 'rb') as f:
        data = f.read()
    encrypted_bytes = cipher.encrypt(pad(data, AES.block_size))
    enc_file_path = file_path + ".enc"
    with open(enc_file_path, "wb") as f:
        f.write(cipher.iv + encrypted_bytes)
    os.remove(file_path)   # remove raw data after locking
    return enc_file_path
```

GCM mode. After the audit, the engine was extended with a GCM mode. The core of the final method chooses the mode. In GCM mode it creates a fresh random nonce and returns the nonce, the tag and the ciphertext [8].

```
def encrypt_bytes(self, data):
    if self.mode == "gcm":
        cipher = AES.new(self.key, AES.MODE_GCM,
                         nonce=os.urandom(NONCE_BYTES))
        ciphertext, tag = cipher.encrypt_and_digest(data)
        return cipher.nonce + tag + ciphertext
    cipher = AES.new(self.key, AES.MODE_CBC)
    return cipher.iv + cipher.encrypt(pad(data, AES.block_size))
```

Decryption. In the first version, `decrypt_data` read the IV from the start of the file, decrypted the data and removed the padding. If anything failed, it printed an error and returned `False`. In GCM mode, decryption calls `decrypt_and_verify`. This checks the tag before any plaintext is returned, so a changed file gives an error and no output. In the final pipeline, the other components decrypt with a shared library. They never delete the encrypted originals, so decryption can be repeated [8].

Keys. The first version created a random key when none was given. A random key would make the dataset impossible to recover. The final engine therefore raises an error for a missing or wrong-length key. The AES key is read from a local environment file that is kept out of version control. It is separate from the HMAC signing key that Privacy Integration uses [8].

Access control. The method `verify_access(user_role, permission)` of `RBACGateway` returns `True` only if the role is in the policy and has the permission. Otherwise, it prints a denial message and returns `False`.

```
self.roles = {
    "Simulation_Engineer": ["create", "verify", "encrypt"],
    "Privacy_Engineer": ["verify"],
    "ML_Researcher": ["read_processed"]
}
```

Capture and metadata. The capture script starts CARLA in each town and applies the settings of the town profile. It drives the ego vehicle and records the four images and the box labels. Each file is protected as described above. One row is written to the town metadata file for every frame, with the fields in Table 4.6. File names follow patterns such as `rgb_f000224.png.enc`, `depth_f000224.png.enc` and `bbox_f000224.txt`.

Storage overhead. A GCM file is exactly 32 bytes larger than the original. This is the 16-byte nonce plus the 16-byte tag. A CBC file is larger by the 16-byte IV plus 1 to 16 bytes of padding. For the 40,000 encrypted images, the CBC overhead is between 680,000 and 1,280,000 bytes in total. This is under 1.3 MB, which is small compared with the image data.

Implementation challenges

- **Encryption mode.** The proposal specified CBC, and the dataset was captured with it. CBC cannot detect a modified encrypted file by itself [23]. A GCM mode was therefore added, and the manifest gate was enforced for the CBC dataset [8].

- **Key handling.** A group code review found the AES key and the HMAC key written into source files. Both were replaced and moved to environment files. A regression test now fails if an old key appears in the source [8].
- **Testing without the simulator.** The security classes were written without any CARLA imports. The audit and the 26 unit tests therefore run on any machine in seconds.
- **Shared identifiers and column names.** Frame numbers repeat across towns. A town prefix was therefore added to every file name when Utility Evaluation merged the towns. A compliance check also failed at first, because it could not find the metadata columns. The quasi-identifier names were then set to the column names used by this component [8].
- **Label coverage.** In 3,264 frames (32.6%), no object is labeled. Privacy Integration also reported that in 62.3% of the frames with a labeled pedestrian or cyclist, the largest such object was smaller than about 8 x 8 pixels after resizing [8]. These are properties of the captured scenes. They limited the tasks that could be trained on the data.

5.3 Integration with the other components

The component is connected to the three other components through files. Table 5.2 lists them.

Table 5.2: Files exchanged by the Dataset Generation component

File	Direction	Purpose
calibration_stats.json	From Privacy Integration	Privatized statistics used to configure the simulator
Encrypted images (.png.enc)	To all three components	Four images per frame, protected with AES-256
Label files (.txt)	To Utility Evaluation and Privacy Integration	YOLO boxes for cars, pedestrians and cyclists
Manifests (SHA-256)	To all three components	Digests of the plaintext files, one manifest per town
Town metadata files (CSV)	To all three components	Labels, counts, weather, time of day, behavior, town and security field
AES key (environment file)	To the consumers, outside the dataset	Lets the other components decrypt the images

Utility Evaluation decrypted all 40,000 image files of the five towns, and every digest matched its manifest. Its poisoning gate then screened the 10,000 frames and 15,084 boxes. The box count equals the sum of the `Num_Objects` field in the metadata files. This confirms that the labels and the metadata agree. Risk and Compliance runs a manifest gate that re-hashes the decrypted dataset before training. No later stage may use data that fails it [8]. Privacy Integration used the frames for its classifier after it excluded frames with only small targets [8].

The privatized calibration values from Privacy Integration were used to set up the town profiles. The Town01 profile matched the privatized file field by field. The weather shares of the Town01 metadata (1,248 clear, 270 cloudy and 482 rainy frames) equal the privatized weather values [8]. The pipeline runner checks every hand-off file against its contract. A change in a file name or a field is therefore detected at the interface [8].

5.4 Verification and software testing

The component was tested at four levels. The 26 unit tests cover the code. The six-test audit tests the security engine on its own. The dataset analysis checks the generated metadata. The other components carried out the integration checks. Table 5.3 shows how each functional requirement was verified.

Table 5.3: Verification of the functional requirements

ID	Requirement	Verification	Status
FR-01	Several towns	Five towns, 2,000 frames each	Passed
FR-02	Four images, box labels and behavior labels	10,000 metadata rows with all fields; 15,084 boxes confirmed by the poisoning gate [8]	Passed
FR-03	SHA-256 digest and manifest	All 40,000 decrypted files matched their manifest [8]	Passed
FR-04	AES-256 encryption of every file	Delivered dataset decrypted by two components; audit Tests 1 to 4 and 6 on the GCM engine	Passed
FR-05	Role-based access control	Policy implemented; allow and deny cases are not part of the six-test audit	Partly verified
FR-06	Structured dataset with metadata	Five town metadata files with 2,000 rows each	Passed
FR-07	Use of privatized calibration statistics	Town01 profile matched the privatized file [8]	Passed
FR-08	Detect modified files	Manifest gate for the CBC dataset; audit Test 5 for GCM	Passed

FR-05 is marked as partly verified on purpose. The role policy exists and is applied in the pipeline. The audit does not yet include tests that show a denial for each wrong role and permission. Adding these tests is the first item of future work in Section 9.4. Table 5.4 lists the test cases of the security audit. It shows the input, the steps, the expected result and the actual result.

Table 5.4: Test cases of the security audit

Test	Input	Steps	Expected	Actual
1	4 KB random bytes	Encrypt, decrypt, compare with the original	Identical bytes	Identical
2	One plaintext	Encrypt 30 times, collect nonces and ciphertexts, count unique values	30 and 30 unique	30 and 30
3	200,000 random bytes	Encrypt, count each byte value, compute chi-square	Below 293.25	272.59
4	Two 4 KB inputs, one bit apart	Encrypt both, XOR the ciphertexts, count differing bits	About 50%	50.0%
5	One encrypted file	XOR one byte with 0xFF, attempt decryption	Rejected, no output	MAC check failed
6	6,093 encrypted images	Read the first 8 bytes, compare with the PNG signature	No matches	0 matches

5.5 Deployment and operation

In the intended deployment, a simulation engineer runs the capture script in each town with the Simulation_Engineer role. Each file is hashed, encrypted and written together with its metadata row. The encrypted folders and the manifests are then copied to the storage that the other components use. The key stays in a local environment file and is delivered separately from the data. A copy of the storage alone is therefore not enough to read the images. A privacy engineer verifies batches before processing. An ML researcher uses only processed datasets.

The group runs the whole pipeline as command-line stages joined by a pipeline runner. A command that checks the environments, inputs and keys runs before each stage starts [8]. The audit can be repeated at any time. A new town or sensor can be added without changing the security classes, because they work on files and not on image content. A change to the calibration statistics needs a new capture, because the privatized values set up the scenes [8].

6 RESULTS AND VALIDATION

6.1 Research results

This section reports the results of the six-test security audit of the GCM engine. It also gives the evidence for the integrity of the delivered CBC dataset. Figure 6.1 shows the results of Tests 2, 3 and 4 and the folder scan.

Test 1: Round-trip correctness

A 4 KB file was encrypted and decrypted, and the recovered bytes were compared with the original.

Table 6.1: Round-trip correctness results

Metric	Result
Decryption reported success	True
Recovered bytes identical to original	True
Encryption time	14.914 ms
Decryption time	25.931 ms

The original data was restored exactly, so encryption does not damage dataset images. The times include reading and writing the file, and they come from a single run. They are small enough that the security layer does not slow the pipeline in a noticeable way.

Test 2: Nonce uniqueness

The same plaintext was encrypted 30 times with the same key.

Table 6.2: Nonce uniqueness results

Metric	Result
Encryptions of identical plaintext	30
Unique nonces	30
Unique ciphertexts	30

All 30 nonces and all 30 ciphertexts were different. A person who sees two encrypted files cannot tell whether they hold the same image. The nonce is 128 bits long and random. For 6,093 files, the chance that any two nonces collide is about 6,093 squared divided by 2 to the power 129. This is about 5 x 10^-32, which is negligible. Thirty runs is a small sample. The test supports nonce uniqueness but does not prove it alone. The guarantee comes from the random nonce generation in the library.

Test 3: Ciphertext byte distribution

A 200,000-byte file was encrypted and the frequency of each byte value in the ciphertext was counted.

Table 6.3: Ciphertext byte distribution results

Metric	Result
Ciphertext sample size	200,000 bytes
Expected count per byte value	781.25
Standard deviation of counts	28.84
Chi-square statistic (df = 255)	272.59
5% critical value (df = 255)	about 293.25
Approximate p-value	about 0.21

The statistic of 272.59 is close to the expected value of 255. It is below the 5% critical value of about 293.25. The p-value of about 0.21 means that this test cannot tell the ciphertext apart from uniformly random bytes.

Test 4: Bit difference between encryptions

Two 4 KB inputs that differ in exactly one bit were encrypted, and the two ciphertexts were compared bit by bit.

Table 6.4: Bit difference results

Metric	Result
Bits compared	32,768
Bits different	16,384
Percentage different	50.0%

The ciphertexts differ in 50.0% of their bits. This is what unrelated random data gives. The result needs careful reading. GCM is built on counter mode. A one-bit change in the plaintext under the same nonce would change only one ciphertext bit. The 50% appears here because each encryption uses its own fresh random nonce. The test therefore shows that similar files give unrelated-looking ciphertexts in practice. It should not be described as the avalanche effect of a block cipher.

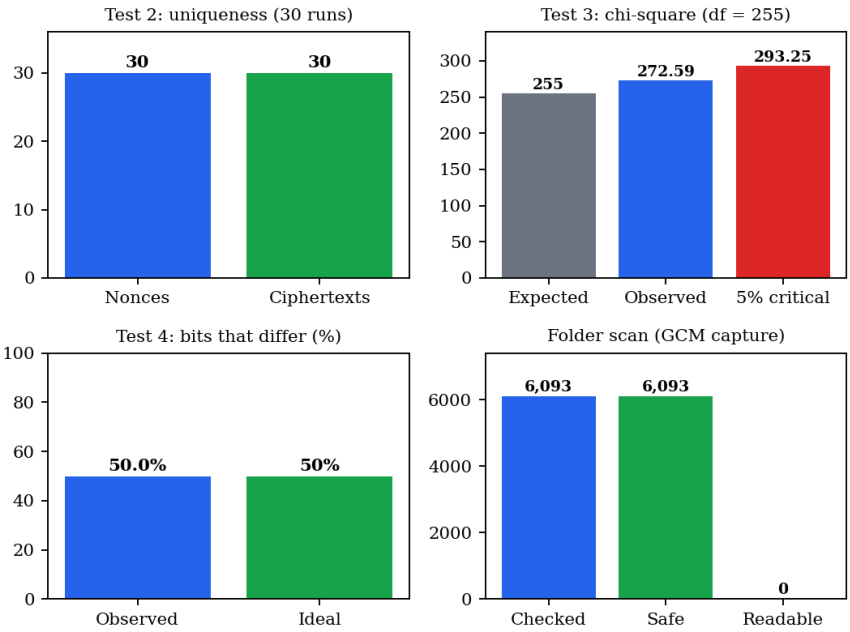


Figure 6.1: Results of audit Tests 2, 3 and 4 and the real folder scan

Test 5: Tamper detection

One byte inside an encrypted file was changed with an XOR of 0xFF, and decryption was attempted.

Table 6.5: Tamper detection results

Metric	Result
Decryption reported success	False
Error raised	MAC check failed
Output file produced	None

The modified file was rejected and no plaintext was written. This is the main practical gain of the GCM mode. A changed file cannot silently damage training data. The delivered CBC dataset has no tag, so the audit reports this test as not applicable for CBC [8]. For the CBC dataset, the manifests play this role.

Test 6: Scan of the real capture folder

The audit scanned every encrypted file in the real Town01 GCM capture. It compared the first eight bytes with the PNG signature. A file that still starts with the PNG signature would be readable without the key.

Table 6.6: Real capture folder validation results

Metric	Result
Encrypted files checked	6,093
Files with no PNG header (safe)	6,093
Files with readable PNG header (vulnerable)	0

All 6,093 files were stored in encrypted form, and none showed a readable header. The scan shows that real output was protected. It does not measure cryptographic strength. That is the purpose of Tests 1 to 5.

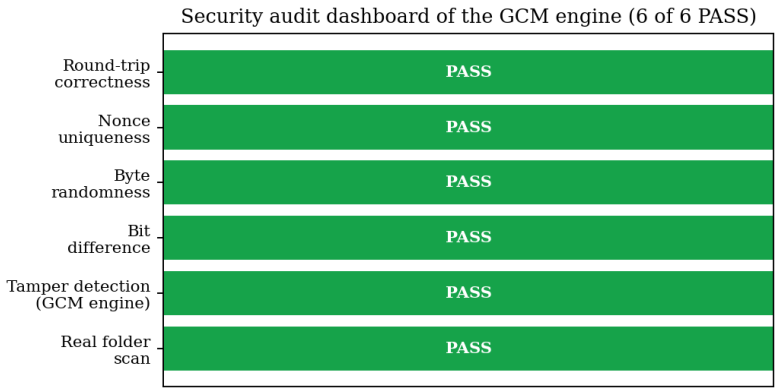


Figure 6.2: Security audit dashboard of the GCM engine (6 of 6 tests passed)

Table 6.7: Summary of audit results for the GCM engine

Property	Key evidence	Outcome
Round-trip correctness	Bytes identical after decryption	Pass
Nonce uniqueness	30 of 30 unique nonces and ciphertexts	Pass
Byte randomness	Chi-square 272.59 (critical 293.25)	Pass
Bit difference	50.0% of bits differ	Pass
Tamper detection	MAC check failed on modified file	Pass
Real folder	0 of 6,093 encrypted files readable	Pass

Integrity of the delivered CBC dataset

The delivered dataset was encrypted in CBC mode, so its integrity rests on the SHA-256 manifests. Table 6.8 shows the evidence reported by the components that used it [8].

Table 6.8: Integrity evidence for the delivered CBC dataset

Measure	Result
Encrypted image files decrypted by Utility Evaluation	40,000
Decrypted files whose digest matched the manifest	40,000 of 40,000
Frames and boxes screened by the poisoning gate	10,000 frames and 15,084 boxes
Sum of Num_Objects in the five metadata files	15,084 (equals the box count)
Audit tamper test for CBC	Not applicable (no authentication tag)

All 40,000 files decrypted correctly and matched their manifests. The round trip therefore works for the delivered dataset, and no file was found changed. The match shows that the files agree with the manifests made at capture time. It does not show who could have changed both a file and its manifest. For this reason, the manifests should be signed or stored apart from the data, as Section 7.5 discusses.

6.2 Product and technical results

The component produced a protected dataset of 10,000 frames. The metadata files of the five towns were loaded into one table. The analysis confirmed 2,000 rows for each town and 10,000 rows in total. Every row has the security field `SHA256+AES256`.

Weather and time-of-day coverage

The weather and time-of-day counts are the same in all five towns (Figure 6.3). Each town has 1,248 clear, 270 cloudy and 482 rainy frames. It also has 288 morning, 230 afternoon, 1,149 evening and 333 night frames. The evening makes up 57.5% of the frames and the afternoon only 11.5%. No frame shows fog. The coverage is the same in every town because every town was set up from the same privatized calibration statistics [8]. As a result, the towns differ in layout and behavior but not in weather.

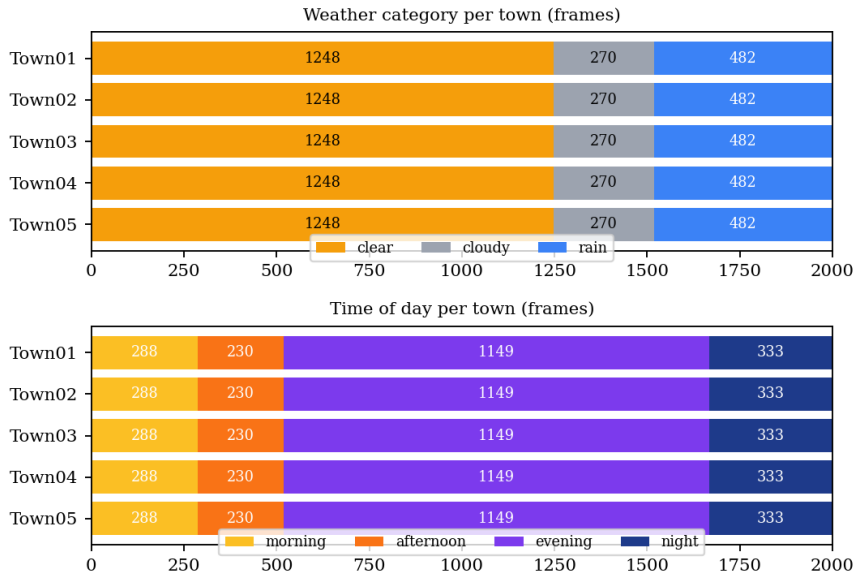


Figure 6.3: Weather category and time of day in each town (number of frames)

Behavior coverage

The behavior labels are not balanced. Table 6.9 and Figure 6.4 show the counts over the whole dataset.

Table 6.9: Distribution of behavior labels over the dataset

Behavior label	Frames	Share of dataset
Standard_Flow	5,691	56.9%
Heavy_Braking	2,694	26.9%
Intersection_Nav	1,414	14.1%
Vehicle_Turning	201	2.0%
Total	10,000	100.0%

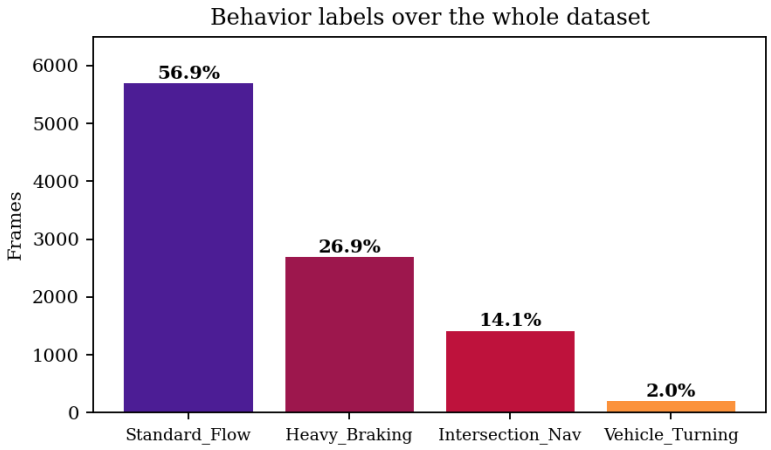


Figure 6.4: Behavior labels over the whole dataset

Standard_Flow makes up 56.9% of the dataset. Heavy_Braking and Intersection_Nav add 26.9% and 14.1%. Vehicle_Turning has only 201 frames, or 2.0%. The largest class is about 28 times the size of the smallest.

Behavior by town

Table 6.10 and Figure 6.5 show how the behaviors are spread over the towns.

Table 6.10: Behavior labels by town (frames and share of the 2,000 frames of each town)

Town	Standard flow	Heavy braking	Intersection	Vehicle turning
Town01	1,241 (62.1%)	598 (29.9%)	117 (5.9%)	44 (2.2%)
Town02	1,120 (56.0%)	691 (34.6%)	90 (4.5%)	99 (5.0%)
Town03	921 (46.1%)	520 (26.0%)	550 (27.5%)	9 (0.5%)
Town04	1,454 (72.7%)	77 (3.9%)	443 (22.2%)	26 (1.3%)
Town05	955 (47.8%)	808 (40.4%)	214 (10.7%)	23 (1.2%)

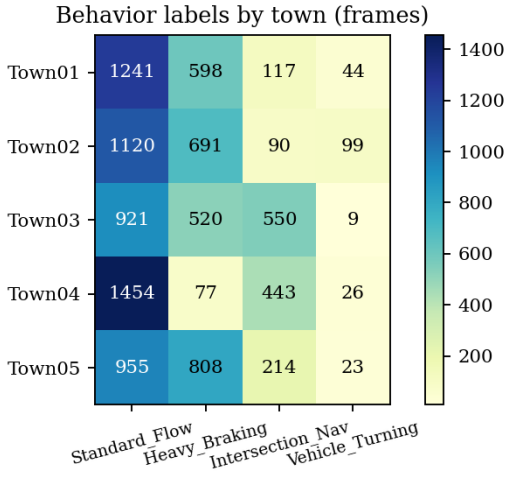


Figure 6.5: Behavior labels by town

Three points stand out. First, Town04 is dominated by Standard_Flow (72.7%) and has only 77 Heavy_Braking frames (3.9%). This fits its long highway. Second, Town03 has the most Intersection_Nav frames (550, or 27.5%) but only 9 Vehicle_Turning frames (0.5%). Third,

Town05 has the most Heavy_Braking frames (808, or 40.4%). The towns therefore differ in layout and also in the behavior they add.

Diversity of behaviors

To compare the diversity of the towns, the normalized Shannon entropy of the behavior labels was calculated from the counts in Table 6.10. It is the entropy of the label distribution divided by the largest possible entropy for four labels. A value of 1 means that all four behaviors are equally common. A value of 0 means that only one behavior occurs. Table 6.11 gives the results.

Table 6.11: Diversity of behavior labels and empty frames by town

Town	Largest class share	Normalized entropy	Frames with no object
Town01	62.1%	0.654	48.5%
Town02	56.0%	0.707	43.1%
Town03	46.1%	0.784	21.8%
Town04	72.7%	0.539	33.4%
Town05	47.8%	0.728	16.5%
All towns	56.9%	0.742	32.6%

The whole dataset has a normalized entropy of 0.742, which is moderate. Town03 is the most balanced town at 0.784. Town04 is the least balanced at 0.539. The weather is the same across towns, so the imbalance is in the behaviors. A model trained on the full dataset will see many examples of normal driving and few examples of turning.

Objects per frame

The number of labeled objects per frame differs strongly between towns. Figure 6.6 shows that the mean rises from 0.74 in Town01 to 2.26 in Town05. The share of frames with no labeled object falls from 48.5% in Town01 to 16.5% in Town05. Table 6.12 gives the distribution over the whole dataset. Town04 has the frame with the most objects, 17. The low count in Town01 matches a warning from the cross-town layer of Utility Evaluation. It found a cyclist share of 0.030 in Town01 against 0.116 elsewhere [8].

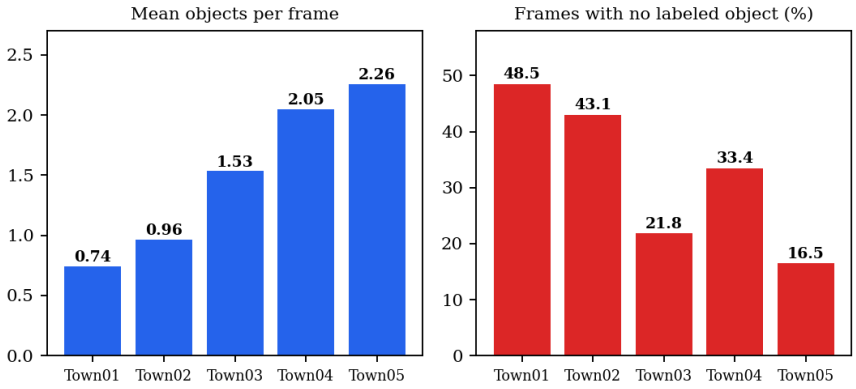


Figure 6.6: Mean labeled objects per frame and share of frames with no labeled object, by town

Table 6.12: Distribution of labeled objects per frame

Labeled objects in the frame	Frames	Share of dataset
0	3,264	32.6%
1	2,932	29.3%
2	1,789	17.9%
3	972	9.7%
4 or more	1,043	10.4%
Total	10,000	100.0%

Cost of the security layer

Table 6.13 summarizes the cost of the security layer.

Table 6.13: Cost of the security layer

Cost	Measured or derived value
Storage overhead per file, GCM	32 bytes (16-byte nonce and 16-byte tag)
Storage overhead per file, CBC	17 to 32 bytes (16-byte IV and 1 to 16 bytes of padding)
Storage overhead for 40,000 CBC images	between 680,000 and 1,280,000 bytes (derived)
Encryption time, 4 KB file	14.914 ms, single run, including file input and output
Decryption time, 4 KB file	25.931 ms, single run, including file input and output
Keys	One 32-byte AES key for all files

Suppose the time of the 4 KB test applied to every file. Encrypting 40,000 images would then take about 10 minutes. Real PNG images are larger than 4 KB, so the real time will be longer. It should be measured on full-size batches. Even so, the cost is small compared with the time needed to run the simulation.

6.3 User or stakeholder validation

The component had no external end users during the project. Validation came from the groups that used or reviewed its outputs.

Supervisors. The supervisors reviewed the work at the end of each sprint. They helped to narrow the scope to the four components [8].

Progress review panels. At the first progress review (PP1), the panel asked for evidence that the component is not vulnerable. The security audit in this thesis was built to answer that comment. At the second review (PP2), the panel gave one comment on this component. Table 6.14 shows it with the response and the concern that remains [8].

Table 6.14: Progress review comments on the component, responses and remaining concerns

Panel	Comment	Response	Remaining concern
PP1	Prove that the component is not vulnerable	Six-test audit of the engine; manifest check of the delivered dataset	Audit has no RBAC denial tests; no formal proof
PP2	How is the original file removed? Is it using crypto-shredding?	The plaintext is deleted with a normal file delete after the encrypted copy is written. This is not crypto-shredding	Encrypt in memory so plaintext is never written; per-batch keys for crypto-shredding

A normal file delete removes only the directory entry. Deleted image bytes may stay on the disk until they are overwritten. The plaintext is deleted, but it is not securely erased. The remaining concern is therefore real. It is part of the limitations in Section 7.5.

Other components as users of the data. Privacy Integration decrypted the dataset, checked each file against the manifest and trained a classifier on it. Its work gave useful feedback on the data. In 62.3% of the frames with a labeled pedestrian or cyclist, the largest such object was smaller than about 8 x 8 pixels after resizing. Frames with only small targets were therefore excluded [8]. Utility Evaluation trained a detector on the synthetic data and on real KITTI images. The synthetic model reached 10.46% of the real-data mAP50. The group target was to stay within 15% of the real-data baseline, so it was not met [8]. This result is about the realism and variety of the whole dataset, not the security layer. Section 7.5 discusses it.

6.4 Integration results

Table 6.15 summarizes how the other components used the output of this component, as reported in the group report [8].

Table 6.15: Integration results reported by the other components

Interface	What was checked	Outcome
Encrypted images and key	Utility Evaluation decrypted all image files	40,000 files decrypted
Manifests	Every decrypted file compared with its manifest	40,000 of 40,000 matched
Manifest gate	Risk and Compliance re-hashes the decrypted data before training	Part of the pipeline run
calibration_stats.json	Town01 profile compared with the privatized file	Matched field by field
Labels and metadata	Poisoning gate screened 10,000 frames and 15,084 boxes	No structural faults; 2,400 frames excluded by the batch layer
Frames for the classifier	Privacy Integration kept frames with usable targets	7,848 usable frames (5,493, 1,176 and 1,179 in the three splits)
Dataset for the detector	Utility Evaluation trained a detector on the synthetic frames	10.46% of real-data mAP50

The integration shows that the security layer did not block the other components. The party that held the key could decrypt the files and check them against the manifest. The poisoning gate excluded 24 batches of 100 frames. It cannot tell whether a batch was tampered with or

came from an unusual stretch of driving, so it excludes both [8]. The results also show a weakness that the security layer cannot fix. The usefulness of the dataset depends on how well the scenes are designed. Chapter 7 discusses this.

6.5 Summary of findings

- The six audit tests on the GCM engine passed. Encryption restored the data exactly. All 30 nonces and ciphertexts were different. The chi-square statistic was 272.59 against a 5% critical value of about 293.25. 50.0% of ciphertext bits differed. A changed file was rejected with a MAC check failure. None of the 6,093 real encrypted files had a readable PNG header.
- The delivered dataset is encrypted with CBC. All 40,000 of its image files decrypted and matched their manifests. Its integrity therefore rests on the manifests.
- Encryption cost 32 bytes per file in GCM mode. A 4 KB file took 14.9 ms to encrypt and 25.9 ms to decrypt.
- The dataset has 10,000 frames in five towns. It has four images, box labels and labels for weather, time of day and behavior. The weather and time-of-day counts are the same in every town. 57.5% of the frames are in the evening, and none show fog.
- Behavior coverage is unbalanced. Standard_Flow is 56.9% and Vehicle_Turning only 2.0%. A third of the frames (32.6%) have no labeled object. The share varies from 48.5% in Town01 to 16.5% in Town05.
- The progress panel asked whether the plaintext is removed with crypto-shredding. It is not. The plaintext is deleted with a normal file delete. This is listed as a limitation.

7 DISCUSSION

7.1 Interpretation of findings

The results answer the main research question with a yes, within clear limits. For RQ1, every captured file was hashed, encrypted and covered by the access policy at capture time. The simulator was used in the normal way, because the security classes work on files after they are written. For RQ2, the GCM engine passed all six audit tests. The data came back exactly. Identical inputs never gave identical outputs. The ciphertext looked random in the statistical test. A changed file was rejected. No real encrypted file was readable. The delivered dataset is encrypted with CBC. All 40,000 of its files decrypted and matched their manifests. Its integrity is therefore checked by the manifests and not by the encryption mode. For RQ3, the dataset covers five towns with four images per frame. The weather is the same in every town, and the behaviors are unbalanced. The security layer costs a few bytes and a few milliseconds per file.

The results need careful reading. The audit shows that no weakness was found in the properties tested. It does not show that the system cannot be broken. A passing statistical test only shows that no pattern was seen. The strength of the design comes mainly from the standard methods it uses. These are AES-256, GCM and SHA-256, used as their standards describe. The tests mostly show that the implementation did not break the design, for example by reusing a nonce or skipping the tag check.

The choice of mode matters most. With CBC, a modified file may decrypt into damaged data, and the pipeline needs a second check to notice. With GCM, the tag check is part of decryption. The delivered dataset uses CBC. The manifests and the manifest gate therefore carry the integrity check. This works, but it is weaker than a tag in one way. A person who can change a file and its manifest together would not be detected, unless the manifests are signed or stored apart from the data. The SHA-256 manifest is also useful in GCM mode. The tag protects the encrypted file. The manifest identifies the original plaintext, and another component can check it after decryption.

7.2 Comparison with literature baselines and existing solutions

The design agrees with the guidance in the literature. Studies of real attacks on GCM show that nonce reuse is the main danger [14], [15]. A fresh random nonce for every file, and a test that no two are equal, deal with this danger directly. Work on CBC shows that unauthenticated encryption can be attacked through its padding behavior [23]. It also shows that combining encryption and authentication correctly is not easy [24]. This is the reason for the GCM mode. It is also the reason why the CBC dataset must be read together with its manifests.

The security surveys of autonomous driving recommend integrity protection, encryption and access control. They stay at the level of recommendations [6], [7], [9]. This thesis adds a working control inside a synthetic data pipeline, with measured evidence. Plain disk or cloud encryption works differently. This component protects each file at the moment of capture. It records a digest for each file and applies a role policy to pipeline operations. Disk encryption

does none of these things on its own. It is cheaper to set up, however, and the two can be used together.

The component also fills a gap that privacy work leaves open. Differential privacy limits what a model or dataset reveals [30], [31]. The work of Stadler et al. shows why synthetic data needs such formal protection [29]. It does not stop someone from changing the files. The security layer and the privacy layer deal with different threats. The project needs both.

7.3 Research contribution

The main research contribution is a measured evaluation of a security layer inside a synthetic AV data pipeline. The evaluation combines six tests, each aimed at a specific weakness. It also includes an independent integrity check of the delivered dataset by the other components. A second contribution is the careful reading of the bit-difference test. In GCM, the 50% result comes from fresh nonces and not from diffusion inside the cipher. Reporting this avoids a common misreading. A third contribution is the honest account of the CBC dataset. The audit found the limit of CBC, a GCM mode was added, and the remaining gap is stated. A fourth contribution is the coverage analysis of the dataset. It shows that a dataset can be fully protected and still be unbalanced. A security layer does not make a dataset more useful.

7.4 Product contribution

The product contribution is the Dataset Generation component. It captures four images and box labels in five towns. It protects every file, enforces role-based access, and delivers the dataset, manifests and metadata to the other three components. The security classes are independent of CARLA and covered by 26 unit tests. Other pipelines can reuse them. The audit lets any team repeat the checks in seconds. The town metadata files make the dataset easy to analyze.

7.5 Limitations and threats to validity

Integrity of the delivered dataset. The delivered dataset is encrypted with CBC, which has no authentication tag. Its integrity depends on the SHA-256 manifests and the manifest gate [8]. The manifests are kept next to the data. A person who can change both a file and its manifest would not be detected. The GCM mode was tested on a separate Town01 capture of 6,093 images, not on the delivered dataset.

Removal of the plaintext. The plaintext is deleted with a normal file delete after the encrypted copy is written. This removes the directory entry only. The image bytes may stay on the disk until they are overwritten. On solid-state disks, an overwrite may not reach them. The progress panel asked about this. The answer is that it is not crypto-shredding [8]. The plaintext also exists on disk for a short time, because CARLA writes the PNG before it is encrypted. One fix is to encrypt the frames in memory so that the plaintext is never written. Another is to run the capture on an encrypted disk. A separate key for each batch would also let a batch be crypto-shredded by destroying its key.

Key management. The AES key is read from a local environment file that is kept out of version control. The final engine refuses to run without a key. The keys are still not held in a key

management service. One key protects every file, so a leak would expose every file. The group code review found keys in source files before the experiment, and they were replaced [8]. NIST guidance says that keys should be protected separately from the data and managed through their life cycle [35].

Access control. The RBAC gateway receives roles as plain strings. It has no user authentication and no persistent audit log [8]. The policy has no permission for decryption, so whoever holds the key decides who may decrypt. The pipeline wrapper enforces the roles. A person with direct access to the files and the key can bypass the wrapper. The allow and deny cases of the policy are not part of the six-test audit.

Internal and construct validity. The audit tests used pseudo-random bytes and not real PNG images. The nonce test used 30 runs. The bit-difference test measures the effect of fresh nonces. It does not measure diffusion inside the cipher. The timings come from single runs. Statistical tests show that no weakness was seen. They are not a formal security proof and do not replace cryptanalysis. Only a chi-square test was used for randomness. A full suite such as NIST SP 800-22 [32] would check more patterns.

External validity. All data comes from one simulator. The folder scan covered 6,093 files from a Town01 GCM capture, while the delivered dataset holds 40,000 encrypted images. The results show that the layer works as designed. They do not show how it behaves at a much larger scale.

Dataset limitations. The weather and time-of-day counts are the same in all five towns, and no frame shows fog. Every town was set up from the same privatized calibration statistics. The behavior labels are unbalanced, with 201 Vehicle_Turning frames out of 10,000. A third of the frames have no labeled object, and the share varies strongly between towns. Pedestrians are often small in the images. This limited the tasks that Privacy Integration could train [8]. The calibration statistics came from a single source, the Waymo Open Dataset. The dataset was captured at one calibration budget only. The part of the utility gap that comes from calibration noise could therefore not be measured [8]. Utility Evaluation found that a detector trained on the synthetic data reached 10.46% of the real-data mAP50. The group target of 15% was not met [8]. The cause is not the security layer, because decryption restores the images exactly (Test 1 and the manifest match). It lies in the realism and variety of the generated scenes.

Table 7.1 summarizes the main limitations, their effect and the suggested remedy.

Table 7.1: Limitations, their effect and suggested remedies

Limitation	Effect	Suggested remedy
Delivered dataset in CBC, no tag	Changes are found only by the manifests	Move the delivered dataset to GCM; sign the manifests
Plaintext removed with a normal delete	Deleted bytes may be recoverable	Encrypt in memory; encrypted disk; per-batch keys
One key for all files, no key service	A leak exposes every file	Key management service, rotation, per-batch keys [35]
Roles are strings, no authentication	A caller can claim any role	Authenticate users before role checks

Limitation	Effect	Suggested remedy
No persistent access log	Access cannot be audited afterward	Tamper-evident access log
Audit lacks RBAC denial tests	FR-05 is only partly evidenced	Add allow and deny tests for every role
Random-byte inputs, 30 nonce samples	Statistical support only	Real PNG files, larger samples, NIST test suite [32]
Same weather in every town; unbalanced behaviors	Limited variety for training	Capture at several calibration budgets; add targeted scenarios

7.6 Implications

For practitioners, the results suggest that a synthetic dataset should be protected when it is generated, not afterward. The cost is small: a few bytes and a few milliseconds per file. An authenticated mode such as GCM is better than an unauthenticated mode. Nonce uniqueness should be tested and not assumed. If an unauthenticated mode is already in use, a manifest check must take the place of the tag. The manifest should also be protected. Deleting a plaintext file is not secure erasure. The plaintext should not be written to disk at all if this can be avoided.

For the design of synthetic data pipelines, security and data quality are separate goals. A dataset can be fully protected and still not be useful if its scenes lack variety. The same weather, the behavior imbalance and the detector result show this. Teams should review coverage together with security. For auditors and data controllers, a per-file manifest, authenticated encryption and a role policy give evidence. They can check it without trusting the team that produced the data.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component gives a team that builds perception models with simulated data a way to protect and share its datasets with evidence. Every file is hashed and encrypted at capture. Access follows a role policy. The security layer can be tested without the simulator. In other approaches, security is added after generation. Here, protection starts when the data is created.

The project proposal named the following groups of users:

- Autonomous vehicle researchers and AI developers, who need training data they can trust and share.
- Simulation engineers, who generate the data and need a layer that does not disturb capture.
- Cybersecurity engineers, who set up, monitor and audit the controls and the keys.
- Technology companies and the automotive industry, which want to improve models without collecting real data.

8.2 Market and competitive position

The target market is teams that use simulation for perception training. Examples are autonomous vehicle start-ups, automotive suppliers, university labs and simulation service providers [8]. Many of them exchange datasets with partners. They must show that they handled the data securely.

Competing approaches each cover only part of the need. Simulators such as CARLA [4] produce data but do not protect it. Storage and cloud encryption protect files, but they are applied after generation. They give no per-file integrity record or role policy for pipeline steps. Anonymization and privacy tools protect what the data reveals, not the files themselves. This component combines hashing, encryption and role-based access at the point of generation. It can be added to an existing CARLA workflow as a security extension.

8.3 Business or sustainability model

The most realistic model for the framework is an open-source core with paid services [8]. Releasing the security classes and the audit as open source lowers the barrier for research labs and builds trust in the method. It also lets users see what each check does. Companies could pay for integration with their own simulators and pipelines, for support and training, and for an independent audit service. Universities could use the toolkit under academic licenses. Table 8.1 summarizes the main parts of the plan for this component. These parts are plans and have not been tested.

Table 8.1: Main parts of the business model for the component

Element	Plan for the Dataset Generation component
Customer segments	Autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers
Channels	Open-source release of the security classes and the audit, the project website and the research paper
Key partners	Open-source communities of CARLA and PyCryptodome, universities and automotive data teams
Key resources	The capture script, the security classes, the 26 unit tests and the audit
Cost structure	Staff time for integration and support, storage for datasets, compute for large simulations
Revenue or sustainability	Paid integration, support and training; an audit service; academic licenses

The running costs are low. CARLA, PyCryptodome and the analysis libraries are free and open-source. The whole pipeline ran on student hardware [8]. The security layer adds only a few bytes per file, so it does not raise storage or compute cost in a significant way. In the group report, a 4 KB file took about 15 ms to encrypt [8].

8.4 Scalability intellectual property and compliance

Scalability. The security layer works on files. Its cost grows with the number of files and not with image content. It can be applied to more towns and more sensors without changes. Large-scale generation would need parallel simulation, a proper key management service and automatic logging of access decisions.

Intellectual property. The component builds on open-source tools. Any release must respect their licenses and keep their notices. The licenses of CARLA and its assets need to be checked before commercial use. The calibration statistics came from the Waymo Open Dataset. The group used it and KITTI under their non-commercial licenses [8]. A commercial product would therefore need a different data source or a license. The code written for this component belongs to the author and the project, subject to the rules of the university.

Compliance. The component produces technical evidence for compliance. Encryption and integrity protection support GDPR Article 32 on security of processing [3]. The evidence can also be mapped to the NIST Privacy Framework [37]. The decision on legal compliance stays with the data controller. A product would also need to state the limits of the layer clearly, such as the key management and plaintext removal limits in Section 7.5.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to two Sustainable Development Goals [38]. These are the same two as the group [8].

- **SDG 9: Industry, Innovation and Infrastructure.** Target 9.5 asks for stronger research and technology capability. The component gives developers a tested way to share synthetic driving data securely. This supports innovation in the automotive sector.

- **SDG 16: Peace, Justice and Strong Institutions.** The component supports responsible handling of data. It gives auditors evidence that they can check. It also protects data that may later relate to real people.

The link to road safety under SDG 3 and SDG 11 is indirect. Safer sharing of training data may help perception models improve. This thesis did not measure any effect on road safety, and the utility gap would need to be closed first. There are also possible negative effects. Encryption can give a false sense of security if users ignore the limits of key handling and plaintext removal. A lost key would make the dataset permanently unreadable. Large simulations use energy. These risks can be reduced in three ways. The limits can be published together with the evidence. Keys can be backed up under a proper key management policy. Only the data that is needed can be captured.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research built a security layer into the synthetic data generation pipeline of an autonomous vehicle training framework. It tested the layer with repeatable checks. The Dataset Generation component captures four images and box labels for 10,000 frames in five CARLA towns. It computes a SHA-256 digest for every file, applies a role-based access policy and encrypts every file with AES-256. It produced a protected dataset of 40,000 encrypted images, with manifests and a metadata file for each town.

The delivered dataset uses CBC mode, so its integrity rests on the manifests. All 40,000 of its files decrypted and matched their manifests. A GCM mode was added after the audit, and the six-test audit of the GCM engine found no failures. Encryption restored the data exactly. In 30 of 30 encryptions of the same input, the nonces and ciphertexts were different. The chi-square statistic of 272.59 was below the 5% critical value of about 293.25. 50.0% of ciphertext bits differed between two encryptions. A modified file was rejected with a MAC check failure. None of the 6,093 real encrypted files had a readable header. The main finding is that integrity, confidentiality and role-based access can be built in at generation time. Standard tools and a small overhead are enough. The other components could decrypt, verify and use the dataset.

The work also shows limits. The delivered dataset has no authentication tag. The plaintext is removed with a normal delete and is not securely erased. A key service does not manage the keys. The RBAC tests are incomplete. The dataset has the same weather in every town and unbalanced behaviors. A secure dataset is not automatically a useful one.

9.2 Achievement of objectives

Table 9.1 compares the results with the objectives in Section 1.5. The group objective that every generated file is protected was fully achieved [8].

Table 9.1: Achievement of objectives

Objective	Outcome	Status
Configure CARLA capture across towns	Five towns, four images and box labels per frame	Achieved
Generate a labeled dataset and describe its coverage	10,000 labeled frames; weather identical across towns; behaviors unbalanced	Achieved
SHA-256 digest and manifest for every file	All 40,000 decrypted files matched their manifests	Achieved
AES-256 encryption and an authenticated mode	Delivered dataset in CBC; GCM mode added and audited; delivered dataset not yet in GCM	Partly achieved
RBAC for three roles	Policy implemented with deny by default; denial tests not yet in the audit	Partly achieved
Independent audit of the security layer	Six tests on the GCM engine, all passed; CBC tamper test not applicable	Achieved
Integrate with the other components	Privatized statistics used; dataset decrypted, verified and used by all three components	Achieved

9.3 Individual research and product contributions

The individual research contributions of this thesis are:

- A measured evaluation of a security layer inside a synthetic AV data pipeline. It has six tests aimed at specific weaknesses and an independent integrity check by other components.
- A careful reading of the bit-difference result for GCM. It separates the effect of fresh nonces from cipher diffusion.
- An honest account of the CBC dataset, its manifest-based integrity and the path to GCM.
- A coverage analysis of the generated dataset by town, weather, time of day, behavior and objects per frame.

The individual product contributions are:

- The `SecurityEngine` and `RBACGateway` classes. They are independent of CARLA and covered by 26 unit tests.
- The encrypted 10,000-frame dataset with manifests, label files and town metadata files.
- The six-test security audit.
- The interfaces that connect the component to the other three components.

9.4 Future directions

The future work is listed in order of priority.

1. **Move the delivered dataset to GCM.** Re-encrypt the dataset with the GCM mode. Agree the file layout with the other components, which must read the nonce and tag first [8].
2. **Stronger file protection.** Encrypt frames in memory so that the plaintext is never written. Keep keys in a key management service and use a separate key for each batch. A batch can then be crypto-shredded [35], [8].
3. **Complete the audit.** Add tests for the RBAC allow and deny cases of every role. Add a test for a modified file and manifest.
4. **Harden access.** Add user authentication, a tamper-evident access log and a clear decryption permission. Sign the manifests or store them separately.
5. **Better scenes.** Capture at several calibration budgets and vary the weather between towns. Add more turning and braking scenes and pedestrians at a larger image scale. Balance the behaviors [8].
6. **More testing of randomness.** Use real PNG images, larger nonce samples and the NIST statistical test suite [32].
7. **Scale and performance.** Measure hashing and encryption on full-size batches and in parallel simulations.

REFERENCES

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2012, pp. 3354-3361.

[2] P. Sun et al., "Scalability in perception for autonomous driving: Waymo Open Dataset," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," Off. J. Eur. Union, vol. L 119, pp. 1-88, May 2016.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in Proc. 1st Annu. Conf. Robot Learn. (CoRL), ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5] T. Sun et al., "SHIFT: A synthetic driving dataset for continuous multi-task domain adaptation," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022.

[6] C. Gao, G. Wang, W. Shi, Z. Wang, and Y. Chen, "Autonomous driving security: State of the art and challenges," IEEE Internet Things J., vol. 9, no. 10, pp. 7572-7595, May 2022.

[7] Y. Liao et al., "Data security in autonomous driving: Multifaceted challenges of technology, law, and social ethics," World Electr. Veh. J., vol. 16, no. 1, art. 6, 2025.

[8] S. A. S. D. Priyadarshi, S. Yubitha, M. K. S. R. Apeksha, and R. D. P. T. Wijesinghe, "Privacy-enhanced synthetic data generation for autonomous vehicle training: Common integrated solution report (R26-CS-010)," Sri Lanka Inst. Inf. Technol., Colombo, Sri Lanka, Oct. 2026.

[9] G. Lippi, M. Aljawarneh, Q. Al-Na'amneh, R. Hazaymih, and L. D. Dhomeja, "Security and privacy challenges and solutions in autonomous driving systems: A comprehensive review," J. Cyber Secur. Risk Audit., vol. 2025, no. 3, pp. 23-41, 2025.

[10] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in Proc. 29th Int. Conf. Mach. Learn. (ICML), 2012, pp. 1467-1474.

[11] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, arXiv:1708.06733.

[12] S. I. Nikolenko, "Synthetic data for deep learning," 2019, arXiv:1909.11512.

[13] C. Xie, Z. Cao, Y. Long, D. Yang, D. Zhao, and B. Li, "Privacy of autonomous vehicles: Risks, protection methods, and future directions," 2022, arXiv:2209.04022.

[14] N. Ferguson, "Authentication weaknesses in GCM," comments submitted to NIST, 2005.

[15] H. Böck, A. Zauner, S. Devlin, J. Somorovsky, and P. Jovanovic, "Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS," in Proc. 10th USENIX Workshop Offensive Technol. (WOOT), 2016.

[16] M. Cordts et al., "The Cityscapes dataset for semantic urban scene understanding," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 3213-3223.

[17] G. Ros, L. Sellart, J. Materzynska, D. Vazquez, and A. M. Lopez, "The SYNTHIA dataset: A large collection of synthetic images for semantic segmentation of urban scenes," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 3234-3243.

[18] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in Field and Service Robotics. Cham, Switzerland: Springer, 2018, pp. 621-635.

[19] J. Tremblay et al., "Training deep networks with synthetic data: Bridging the reality gap by domain randomization," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), 2018, pp. 969-977.

[20] Secure Hash Standard (SHS), FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[21] Advanced Encryption Standard (AES), FIPS PUB 197, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001.

[22] M. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," NIST Special Publication 800-38A, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001.

[23] S. Vaudenay, "Security flaws induced by CBC padding: Applications to SSL, IPSEC, WTLS...," in Advances in Cryptology - EUROCRYPT 2002, ser. Lecture Notes Comput. Sci., vol. 2332. Berlin, Germany: Springer, 2002, pp. 534-545.

[24] M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in Advances in Cryptology - ASIACRYPT 2000, ser. Lecture Notes Comput. Sci., vol. 1976. Berlin, Germany: Springer, 2000, pp. 531-545.

[25] D. A. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in Progress in Cryptology - INDOCRYPT 2004, ser. Lecture Notes Comput. Sci., vol. 3348. Berlin, Germany: Springer, 2004, pp. 343-355.

[26] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2007.

[27] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," Computer, vol. 29, no. 2, pp. 38-47, Feb. 1996.

[28] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278-1308, Sep. 1975.

[29] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in Proc. 31st USENIX Secur. Symp., 2022, pp. 1451-1468.

[30] M. Abadi et al., "Deep learning with differential privacy," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), 2016, pp. 308-318.

[31] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," Found. Trends Theor. Comput. Sci., vol. 9, no. 3-4, pp. 211-407, 2014.

[32] A. Rukhin et al., "A statistical test suite for random and pseudorandom number generators for cryptographic applications," NIST Special Publication 800-22 Rev. 1a, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2010.

[33] S. Hernan, S. Lambert, T. Ostwald, and A. Shostack, "Uncover security design flaws using the STRIDE approach," MSDN Mag., Nov. 2006.

[34] CARLA Team, "CARLA documentation: Maps," [Online]. Available: https://carla.readthedocs.io

[35] E. Barker, "Recommendation for key management: Part 1 - General," NIST Special Publication 800-57 Part 1 Rev. 5, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2020.

[36] PyCryptodome Project, "PyCryptodome documentation," [Online]. Available: https://www.pycryptodome.org

[37] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[38] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," General Assembly Resolution A/RES/70/1, 2015.

GLOSSARY

Authenticated encryption (AEAD). Encryption that also proves the data was not changed. A change makes decryption fail.

Authentication tag. A 16-byte value that GCM computes over the ciphertext and checks during decryption.

Cipher Block Chaining (CBC). A block cipher mode that gives secrecy but no built-in detection of changes. The delivered dataset uses it.

Galois/Counter Mode (GCM). A block cipher mode that encrypts with a counter and adds an authentication tag.

Initialization vector (IV). A random 16-byte value stored at the start of a CBC file so that equal inputs give different outputs.

Nonce. A number used once per encryption. In GCM it must never repeat under the same key.

PKCS7 padding. A rule that adds 1 to 16 bytes to the end of the data so that its length fits the AES block size.

SHA-256 digest. A 256-bit fingerprint of a file. Any change to the file changes the digest.

Hash manifest. A list of file names and their SHA-256 digests that is kept for a town.

Manifest gate. The check in Risk and Compliance that re-hashes the decrypted dataset and stops the run if a digest does not match.

Role-based access control (RBAC). A policy in which permissions are given to roles and users act through roles.

Least privilege. The rule that each role gets only the permissions it needs.

Deny by default. The rule that anything not explicitly allowed is refused.

Crypto-shredding. Making data unreadable by destroying the key that encrypts it. The component does not do this.

Chi-square test. A statistical test that compares observed counts with expected counts. Here it checks whether byte values in a ciphertext are evenly spread.

YOLO label format. A text format with one line per object: a class and the normalized center, width and height of its box.

Ego vehicle. The simulated vehicle that carries the sensors and drives through the town.

Behavior label. The driving behavior of the ego vehicle when a frame was captured, for example Heavy_Braking.

Plaintext. The original, unencrypted data.

APPENDICES

Appendix A Extended results

This appendix gives the raw audit values and the data behind the coverage figures in more detail than Chapter 6. Table A.1 lists the raw values of the security audit.

Table A.1: Raw values of the security audit and the manifest check

Measure	Value
Round-trip: recovered bytes identical	True
Round-trip: encryption time (4 KB)	14.914 ms
Round-trip: decryption time (4 KB)	25.931 ms
Nonce test: encryptions / unique nonces / unique ciphertexts	30 / 30 / 30
Byte distribution: sample size	200,000 bytes
Byte distribution: expected count per value	781.25
Byte distribution: standard deviation of counts	28.84
Byte distribution: chi-square (df = 255)	272.59
Byte distribution: 5% critical value / p-value	about 293.25 / about 0.21
Bit difference: bits compared / bits different	32,768 / 16,384 (50.0%)
Tamper test: result	MAC check failed; no output file
Folder scan: files checked / safe / readable	6,093 / 6,093 / 0
Delivered dataset: files decrypted / digests matching	40,000 / 40,000

Table A.2 shows the first three records of the Town01 metadata file. It illustrates the metadata schema in Table 4.6. The frame numbers differ by 16.

Table A.2: First three records of the Town01 metadata file

RGB_File	BBox_File	Objects	Weather	Time	Event_Type
rgb_f000224.png.enc	bbox_f000224.txt	0	rain	evening	Standard_Flow
rgb_f000240.png.enc	bbox_f000240.txt	0	rain	morning	Standard_Flow
rgb_f000256.png.enc	bbox_f000256.txt	0	rain	afternoon	Standard_Flow

Table A.3 gives the number of frames of each weather and time-of-day combination in each town. The totals for weather category and time of day are the same in all towns. The combinations differ slightly.

Table A.3: Frames per weather label and town

Weather label	Town01	Town02	Town03	Town04	Town05	Total
clear_morning	194	177	176	184	182	913
clear_afternoon	151	145	144	143	142	725
clear_evening	703	709	718	731	731	3,592
clear_night	200	217	210	190	193	1,010
cloudy_morning	34	43	39	36	39	191
cloudy_afternoon	31	21	30	35	32	149

Weather label	Town01	Town02	Town03	Town04	Town05	Total
cloudy_evening	155	169	153	155	155	787
cloudy_night	50	37	48	44	44	223
rain_morning	60	68	73	68	67	336
rain_afternoon	48	64	56	52	56	276
rain_evening	291	271	278	263	263	1,366
rain_night	83	79	75	99	96	432

Table A.4 gives the number of frames of each behavior label in each town. It is the data behind Figure 6.5.

Table A.4: Number of frames per behavior label and town

Town	Standard_Flow	Heavy_Braking	Intersection_Nav	Vehicle_Turning
Town01	1,241	598	117	44
Town02	1,120	691	90	99
Town03	921	520	550	9
Town04	1,454	77	443	26
Town05	955	808	214	23
Total	5,691	2,694	1,414	201

Table A.5 gives the expected result of every role and permission pair under the access policy. The audit does not test each of these cases yet. They are the planned denial tests.

Table A.5: Expected outcome of each role and permission pair

Role	create	verify	encrypt	read_processed	Other
Simulation_Engineer	Allow	Allow	Allow	Deny	Deny
Privacy_Engineer	Deny	Allow	Deny	Deny	Deny
ML_Researcher	Deny	Deny	Deny	Allow	Deny
Unknown role	Deny	Deny	Deny	Deny	Deny

Table A.6 describes the steps of the security audit so that another person can repeat it.

Table A.6: Steps of the security audit

Step	Action	Output
1	Create the SecurityEngine with a random key that exists only in memory	Engine ready
2	Run Test 1 on a 4 KB random file	Match result and timings
3	Run Test 2: encrypt one plaintext 30 times	Counts of unique nonces and ciphertexts
4	Run Test 3 on a 200,000-byte file	Byte counts, chi-square statistic
5	Run Test 4 on two inputs one bit apart	Number and share of differing bits
6	Run Test 5: change one byte and decrypt	MAC check result, no output file
7	Run Test 6: scan the real encrypted folder with the project key	Counts of safe and readable files; no plaintext written
8	Save charts and results	Result file and image files

Appendix B Academic integrity and AI use disclosure

Generative AI was used while preparing this thesis, in line with the module policy on AI use.

Tool. Claude, a large language model by Anthropic.

Use. The tool helped to structure the thesis using the module template and a peer thesis. It helped to draft and edit text based on the author's proposal, notebooks, code, metadata files and the group report. It helped to create the diagrams and charts from the author's own results. It helped to check references against published records. It also helped to compute derived values such as the chi-square critical value, the p-value, percentages and entropy.

Verification. The author checked the numbers, citations and claims in the thesis against the metadata files, the code, the audit results and the group report.

Responsibility. The design of the component, the experiments, the results and the conclusions are the responsibility of the author.